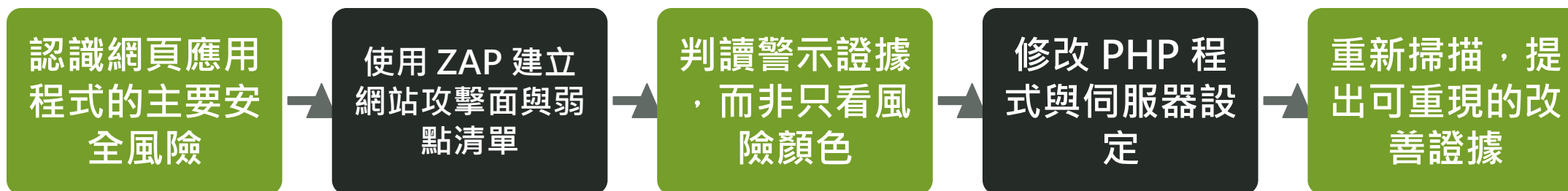


這門課要完成什麼？

認識網頁應用程式的主要安全風險



使用 ZAP 建立網站攻擊面與弱點清單
修改 PHP 程式與伺服器設定

判讀警示證據，而非只看風險顏色
重新掃描，提出可重現的改善證據

- 能說明弱點、威脅、風險與控制措施的差異
- 能以 OWASP Top 10:2025 建立風險共同語言
- 能安全設定 ZAP 的範圍、模式與掃描原則
- 能分析警示的 URL、參數、證據與修補建議
- 能以「掃描—修補—重掃」完成驗證

一個 Web 請求經過哪些信任邊界？

瀏覽器輸入與前端檢查皆不可信任



HTTP 參數、標頭、Cookie 與檔案都可能被修改 | 伺服器端必須重新驗證身分、
授權與輸入 | 資料庫及外部 API 也需要最小權限與輸出限制

Web 攻擊面從哪裡出現？

- 公開頁面、登入頁、管理後臺與隱藏路徑
- URL 查詢參數、表單、JSON、XML、
Cookie 與 HTTP 標頭
- 檔案上傳、匯入、下載、預覽與轉檔功能
- API、WebSocket、第三方登入與外部服務整合
- 伺服器、框架、套件、容器與雲端設定

弱點、威脅與風險

- 弱點 Vulnerability：可被利用的設計、程式或設定缺失
- **威脅 Threat：**
可能利用弱點造成損害的來源或事件
- **利用 Exploit：**
使弱點產生實際影響的方法或程式
- **風險 Risk：**
發生可能性與衝擊程度的綜合判斷
- 控制措施 Control：預防、偵測、回應或復原的措施

Web 弱點常見根因

- 把 HTML 屬性或 JavaScript 檢查誤當成安全控制
- 伺服器端未重新檢查型別、長度、範圍、格式與允許值
- SQL、命令、樣板或路徑以字串拼接
- 僅判斷是否登入，未檢查資源擁有者與角色
- 錯誤訊息、除錯模式、備份檔與預設帳號外洩資訊
- 套件、建置流程與部署環境缺少持續管理

安全邊界在伺服器，不在瀏覽器

使用者可控制的環境

- HTML 屬性可以修改
- JavaScript 可以停用
- 隱藏欄位可以改值
- HTTP 請求可以自行組裝

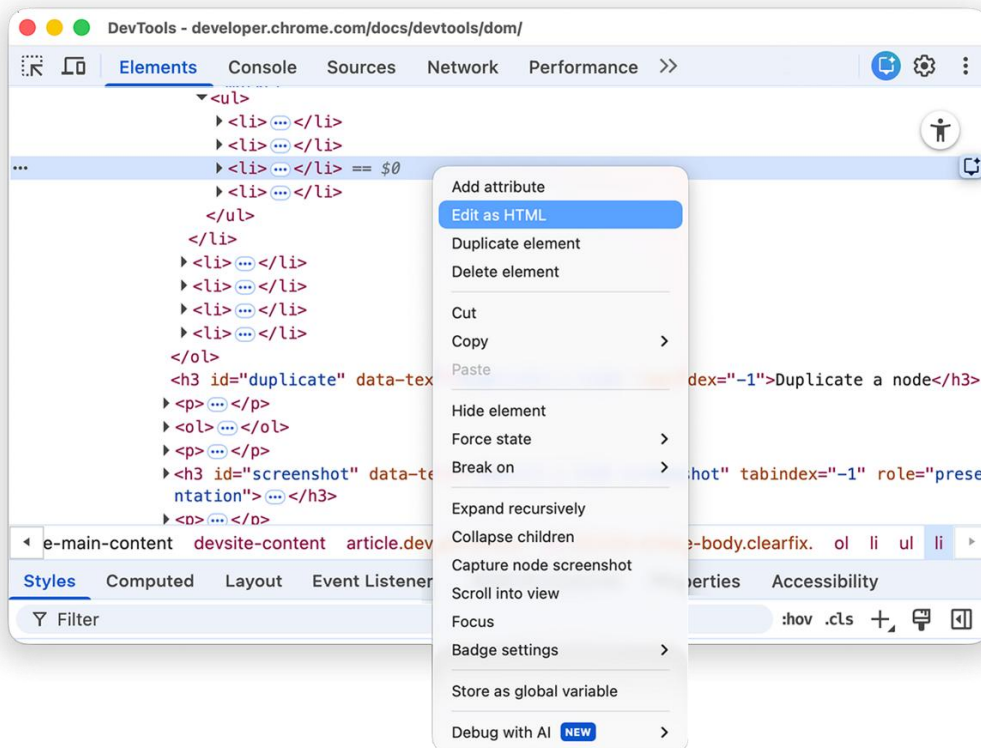


伺服器必須重新判斷

- 解析資料型別
- 檢查長度與範圍
- 核對允許值與業務規則
- 驗證身分、**授權**與資源擁有者

前端驗證改善操作體驗；伺服器端驗證才是安全控制。

不用 ZAP | 開發者工具就能移除 HTML 限制



操作步驟

- 1 按 F12，於 Elements / 元素面板找到輸入欄位
- 2 移除 min、max、required、pattern、disabled 或 readonly
- 3 修改 hidden 欄位或數值後重新送出表單

```
<input type="number" name="quantity" min="1" max="5">
```

改送：
quantity=-1

若伺服器仍接受 -1，代表限制只存在於瀏覽器。

操作畫面來源：Chrome for Developers

不用 ZAP | 網址列與主控台也能送出異常值

GET 參數

把 ?id=2 改成 ?id=999、?id=-1 或其他人的識別碼

瀏覽器主控台

```
fetch('/register', {method:'POST', body:'quantity=-1'})
```

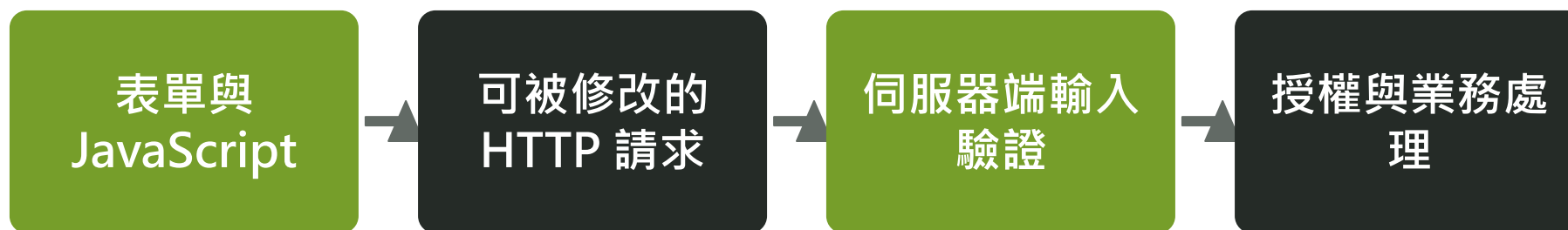
Network 面板

複製請求為 fetch，再修改欄位、Cookie 或標頭後送出

安全測試的單位是 HTTP 請求，不是畫面上的輸入框。

JavaScript 驗證可以被完全跳過

攻擊者可停用 JavaScript、使用代理伺服器，或自行建立請求。



無效資料：回傳 400 / 422，不進入後續流程

接受資料之前完成檢查；失敗時停止流程。

伺服器端輸入驗證至少要檢查八件事

是否存在

必填欄位不可缺少；區分缺少、空字串與 null

資料型別

整數、布林、日期、陣列與物件必須明確解析

長度

名稱、備註、Token 與陣列筆數設定上下限

數值範圍

quantity 只能為 1–5，不能只依 HTML min / max

格式

日期、課程代碼與識別碼應完整符合規則

允許值

role、status、sort 僅能來自伺服器定義的集合

欄位關係

開始日早於結束日；折扣不得超過訂單金額

檔案屬性

大小、實際 MIME、內容、重新命名及隔離儲存

語法正確，不代表在業務上可以接受

語法驗證 Syntactic Validation

- quantity 能解析成整數
- 日期符合 YYYY-MM-DD
- 課程代碼符合完整格式
- JSON 欄位符合預期型別

語意驗證 Semantic Validation

- quantity 介於 1 與剩餘名額
- 開始日期早於結束日期
- 課程目前允許該角色報名
- 折扣、金額與狀態符合流程規則

兩層都要通過；任一失敗即拒絕請求。

先正規化與解析，再執行驗證

驗證後不可再次用不同方式解碼，避免同一內容出現多種表示。



任何步驟失敗：停止處理並回傳一致的錯誤結果

接受資料之前完成檢查；失敗時停止流程。

輸入驗證很重要，但不能取代情境化防護

風險	主要控制措施	輸入驗證的角色
SQL Injection	參數化查詢	限制型別、長度與允許值，降低攻擊面
XSS	依輸出情境編碼 / 必要時 Sanitizer	確認結構化欄位；自由文字仍須安全輸出
Command Injection	避免 Shell、使用安全 API	目的地主機、選項與參數採允許清單
IDOR / 越權	伺服器端授權與擁有者檢查	識別碼格式正確仍不代表有權存取
檔案上傳	隔離、重新命名、不可執行	檢查大小、內容、實際 MIME 與允許類型

驗證失敗時要明確拒絕，不能悄悄放行

- 在任何資料庫、檔案、命令或業務操作之前完成驗證
- 回傳一致且可判讀的 400 或 422；不要自動改成另一個敏感值
- 欄位錯誤訊息要能協助修正，但不得洩漏 SQL、路徑或堆疊資訊
- 不在日誌保存密碼、Token、完整 Cookie 或敏感 Payload
- 對固定選項出現未定義值的情況留下安全事件紀錄
- 修補後以完全相同的異常請求重測，確認資料沒有被寫入

SAST：不執行程式，檢查原始碼或位元碼



SCA：盤點第三方元件與已知弱點

IAST：在執行環境內觀察資料流

DAST：從外部對執行中的網站送出請求

人工測試：驗證商業邏輯與工具難以判定的問題

弱點掃描與滲透測試不相同

- **弱點掃描：**

重視廣度、一致性、重複執行與初步盤點

- **滲透測試：**

重視弱點驗證、攻擊鏈、實際影響與情境推演

- **工具限制：**

未被探索的頁面、角色與流程可能完全漏測

- **判讀責任：**

高風險警示仍須人工確認，不應直接視為入侵成功

- **完整評估：**

結合自動化掃描、人工測試與程式碼檢查

DAST 能直接觀察執行中的 HTTP 行為

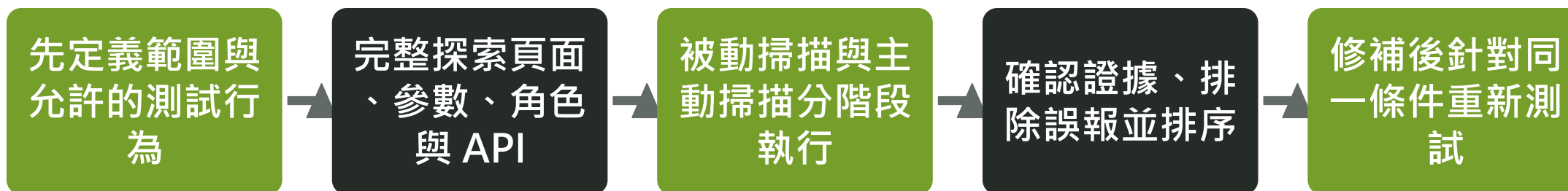
- 不必取得原始碼，也能從執行中的網站觀察攻擊面
- 可檢查請求、回應、Cookie、標頭、參數與伺服器行為
- 能修改瀏覽器送出的內容，驗證使用者端限制是否可繞過
- 可在開發、測試與正式上線前環境重複執行
- 結果通常包含 URL、參數、證據及可重現的改善方向

DAST 的限制

- 未探索到的頁面、角色或狀態就不會被充分測試
- 難以理解所有商業規則與資料所有權
- 供應鏈、原始碼祕密與不可達程式碼需其他工具補足
- 身分驗證流程設定錯誤會讓掃描只停留在登入頁
- 「未發現」不等於「不存在」

安全掃描的閉環流程

先定義範圍與允許的測試行為



完整探索頁面、參數、角色與 API
據、排除誤報並排序

被動掃描與主動掃描分階段執行
修補後針對同一條件重新測試

確認證

OWASP Top 10 是什麼？

- 提供開發人員與 Web 安全工作者的風險認知基準
- 以風險類別整合多個 CWE，而非十個單一弱點
- 適合作為教育、溝通與改善優先順序的起點
- 不等同完整測試清單，也不保證法規符合性
- 最新正式版本為 OWASP Top 10:2025

OWASP Top 10:2025 | A01-A10 一覽

編號	中文名稱	官方英文名稱	常見案例
A01	權限控制失效	Broken Access Control	IDOR、SSRF、路徑穿越、功能越權
A02	安全設定錯誤	Security Misconfiguration	除錯模式、備份檔、缺少標頭、CORS
A03	軟體供應鏈失效	Software Supply Chain Failures	脆弱元件、惡意套件、建置與更新流程
A04	加密機制失效	Cryptographic Failures	明文、弱雜湊、固定金鑰、弱亂數
A05	注入	Injection	SQLi、XSS、Command、CSS、CRLF、XPath
A06	不安全設計	Insecure Design	流程跳步、負數交易、超賣、優惠濫用
A07	身分驗證失效	Authentication Failures	弱密碼、帳號列舉、工作階段固定
A08	軟體或資料完整性失效	Software or Data Integrity Failures	Mass Assignment、反序列化、未驗證更新
A09	安全日誌與告警失效	Security Logging and Alerting Failures	未記錄、未告警、缺乏稽核軌跡
A10	例外狀況處理不當	Mishandling of Exceptional Conditions	Stack Trace、Fail Open、狀態不一致

2025 版的重要變化

- A01 權限控制失效維持第一，並納入 SSRF
- A02 安全設定錯誤由 2021 年第五名升至第二名
- A03 擴大為整體軟體供應鏈失效，不只過時元件
- **A09 強調「告警」：**
只有日誌、沒有告警仍不足
- **新增 A10 例外狀況處理不當**
- **2025 版十類共涵蓋 248 個 CWE**

A01

Broken Access Control

使用者可讀取、修改或執行超出權限的資源

- 登入只證明身分，不能取代每次操作的授權判斷
- 改善重點是伺服器端集中授權、拒絕為預設與最小權限
- 測試時要改變物件編號、角色、HTTP方法與登入狀態

A01 | VulnCampus 可延伸的權限案例

IDOR / BOLA

profile.php、API profile：更改 id 存取他人個資

Path Traversal

download.php?file=../src/db.php 讀取非授權檔案

SSRF

ssrf_demo.php 由伺服器探測內網或本機資源

使用者端控制缺失

hidden_control.php 僅把提權按鈕藏在前端

Switch Fall-through

switch_defect.php 因漏寫 break 進入管理員流程

任意檔案上傳

upload.php 缺少類型、路徑與執行權限限制

A01 案例 | 更改 id 就看到別人的個人資料

實際情境

student01 登入後，把
profile.php?id=2 改成 id=3
；**伺服器**只確認已登入，沒有
確認資料擁有者。

如何驗證

比較不同帳號、不同 id 與 API 回應；HTTP 200 且出現非本人資料，即為可重現證據。

安全改善

查詢條件同時限制資源識別碼與擁有者；管理員功能另做角色授權。

修補證據

修補後以相同帳號與 id 重測，應回傳 403 / 404，且不得洩漏資源是否存在。

IDOR / 越權存取 是什麼？

• 定義：

攻擊者只要修改 URL、參數或物件識別碼，就能存取本來不屬於自己的資料或功能。

• 常見情境：

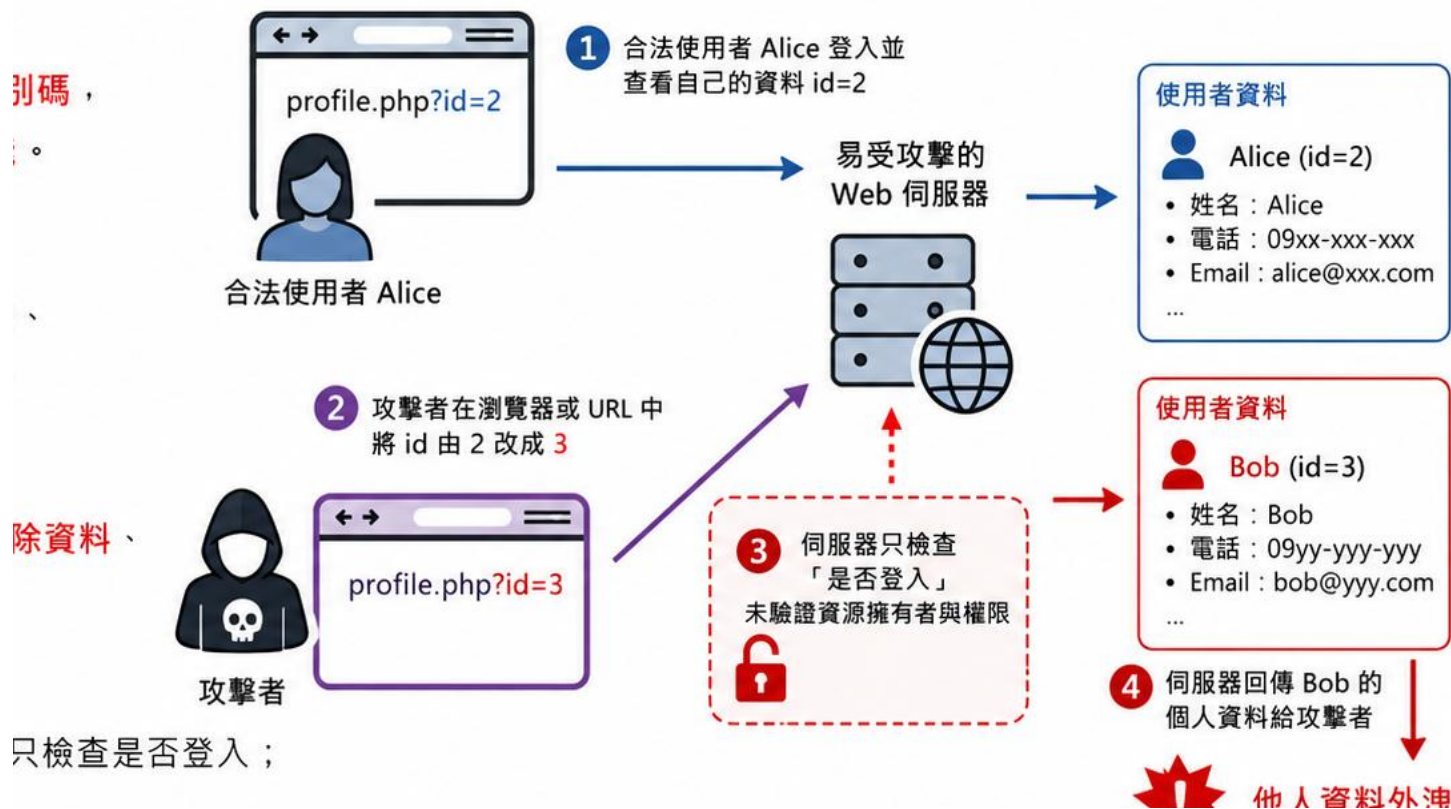
profile.php?id=3、/api/orders/1024、下載別人的檔案、**直接**存取管理功能。

• 可能影響：

個資**外洩**、任意查詢、**未授權**修改、刪除資料、橫向存取其他**帳號**資料。

• 防護重點：

每次存取都驗證資源擁有者與**權限**；使用伺服器端授權判斷與**最小權限**。



重點： 只驗證「有沒有登入」還不夠，還必須驗證「可不可以存取這筆資源」。

A01 案例 | 路徑穿越與 SSRF 不是同一件事

實際情境

Path Traversal 操作本機檔案路徑；SSRF 則讓**伺服器**代替攻擊者發出網路請求。

如何驗證

分別測試 ../、file://、localhost、內網 IP 與重新導向，但僅限授權實驗環境。

安全改善

檔案以識別碼查表；外連採目的地主機允許清單、封鎖私有位址並限制協定。

修補證據

重測時輸入不得控制實際檔案路徑，也不得連線至未核准的目的地。

Path Traversal (路徑穿越) 是什麼？

• 定義：

攻擊者操控檔案路徑，讓程式讀取或存取原本不該碰到的檔案。

• 常見特徵：

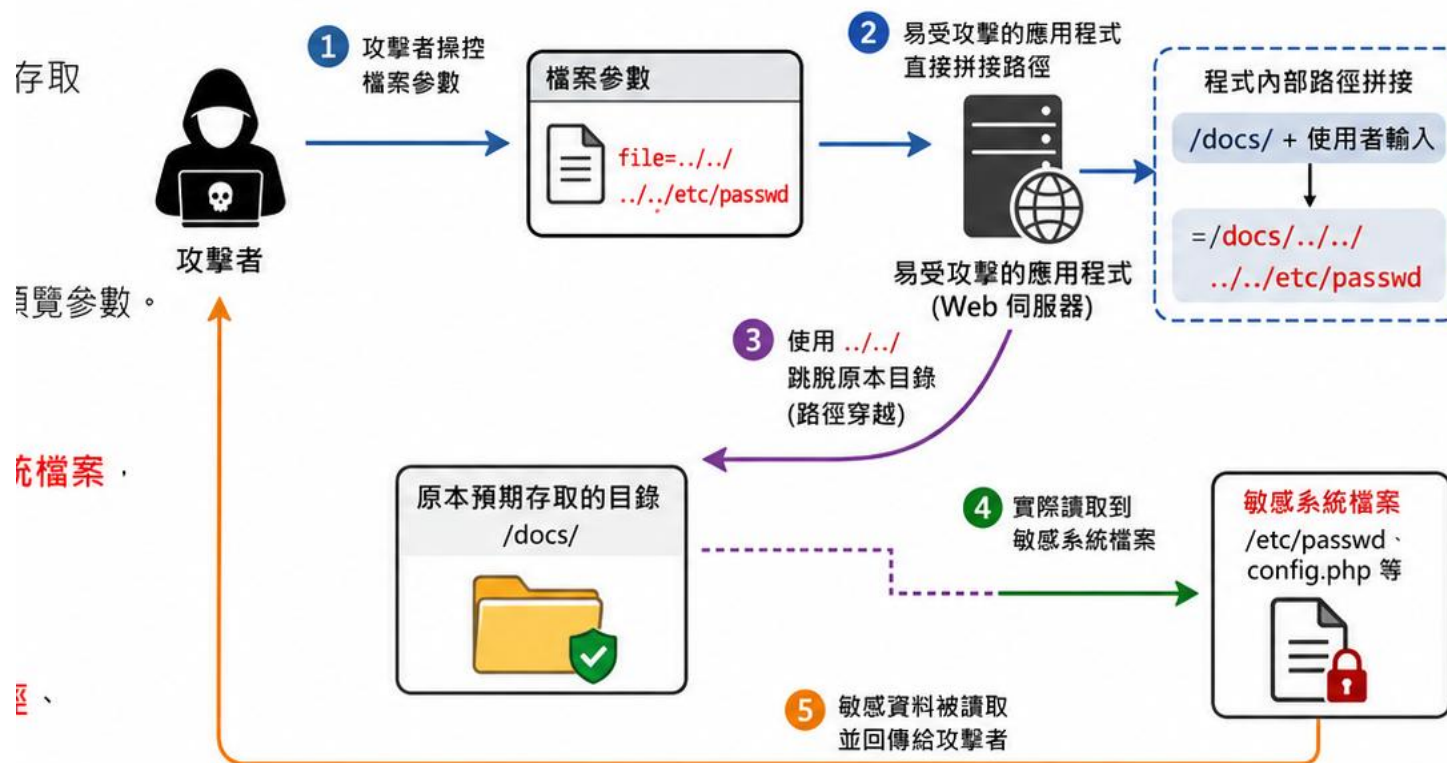
../、..\、URL 編碼變形、檔案下載或預覽參數。

• 可能影響：

讀取設定檔、原始碼、帳密憑證、系統檔案，甚至配合其他漏洞擴大影響。

• 防護重點：

伺服器端允許清單、不要直接拼接路徑、路徑正規化、限定根目錄。



★ **重點：** 只要輸入能改變「實際存取到哪個檔案」，就可能形成路徑穿越。

SSRF (伺服器端請求偽造) 是什麼？

- 定義：

攻擊者操控伺服器去請求原本不應存取的內部或外部資源。

- 常見入口：

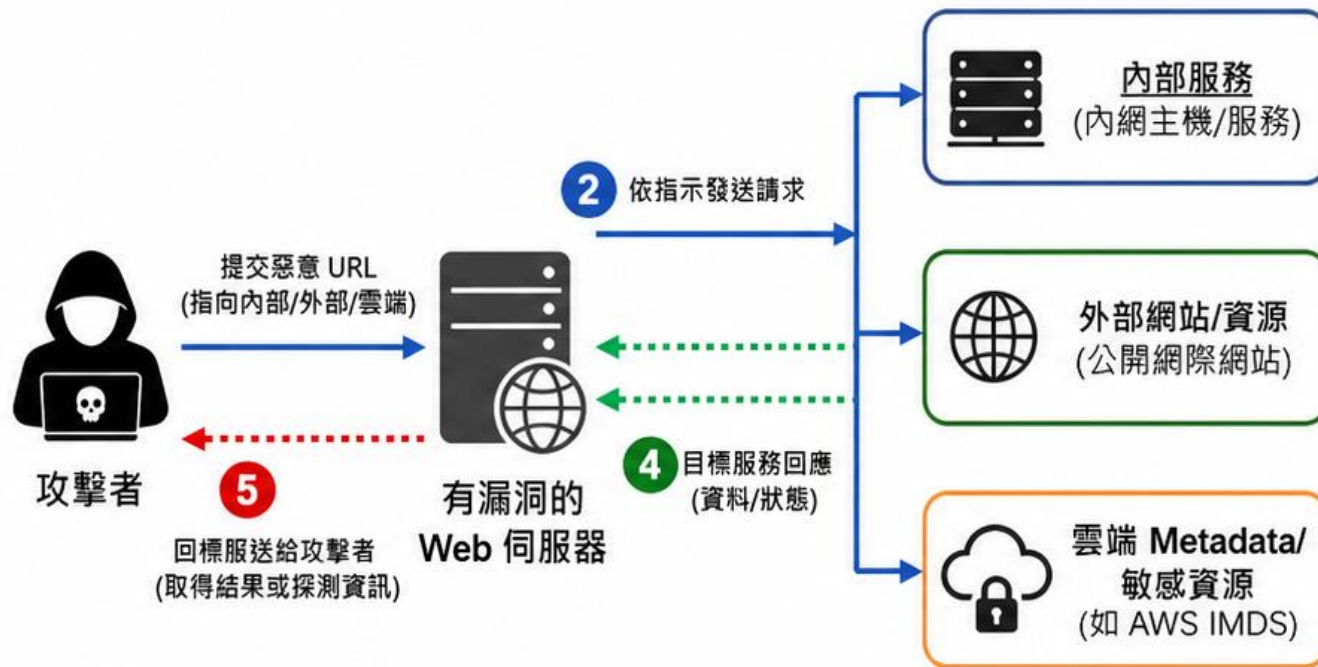
圖片抓取、Webhook、URL 預覽、檔案匯入、API 串接。

- 可能影響：

探測內網、存取 127.0.0.1/localhost、雲端 Metadata、繞過防火牆。

- 防護重點：

URL allowlist、封鎖內網位址、DNS 解析檢查、限制協定與 Port、網路隔離。



★ **重點：** 攻擊者利用網站伺服器代替自己發送請求，可能探測內部服務或存取雲端中繼資料。

A02

Security Misconfiguration

部署、伺服器、框架、雲端或瀏覽器安全設定不足

- ZAP 被動掃描通常能看見安全標頭、Cookie 與資訊洩漏
- 設定錯誤常在測試環境複製到正式環境後持續存在
- 改善需要可重複的強化基準與環境自動檢查

A02 | VulnCampus 的設定錯誤不只一種

除錯資訊

debug.php、詳細錯誤與版本資訊

公開備份檔

db.php.bak、index.php.old
、 backup.zip

安全標頭

HSTS、Referrer-Policy、X-Frame-Options 缺失

Cookie / 快取

HttpOnly、Secure、SameSite 或 Cache-Control 不足

CORS

反射 Origin 且允許 Credentials

目錄與索引

uploads/、robots.txt、sitemap.xml 暴露路徑

A02 案例 | 除錯模式與備份檔形成攻擊地圖

實際情境

公開的 debug.php、.bak、.old、.zip 與版本標頭，讓攻擊者取得路徑、設定、帳密或套件版本。

如何驗證

以 ZAP Sites Tree、被動警示與手動路徑檢查比對公開檔案及 Response Headers。

安全改善

部署時移除備份檔、關閉詳細錯誤、隱藏版本並套用安全標頭基準。

修補證據

修補後路徑回傳 404，錯誤頁不含堆疊、SQL、實體路徑或版本細節。

除錯模式外洩 是什麼？

- **定義：**

網站把**詳細錯誤**、堆疊、**路徑**或設定直接回給使用者。

- **常見情境：**

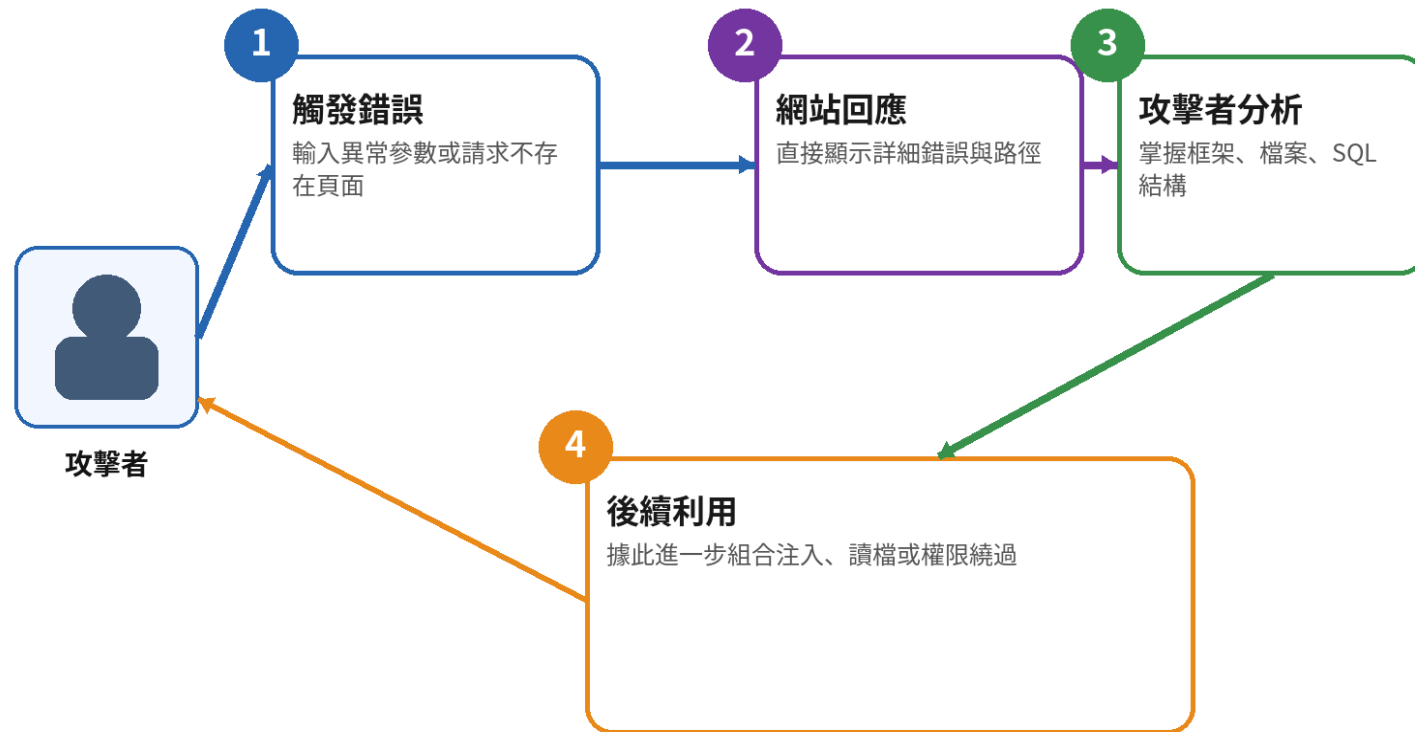
開啟 **debug**、顯示 **stack trace**、**SQL** 錯誤、絕對**路徑**。

- **可能影響：**

攻擊者快速摸清框架、檔案結構、**資料庫**與可利用點。

- **防護重點：**

正式環境**關閉**除錯、集中記錄錯誤、對外只回 **generic message**。



★ **重點：** 錯誤訊息不只是提示，也可能是**攻擊者**的偵察資料。

備份檔外洩 是什麼？

- **定義：**

可透過 Web **直接** 下載 .bak、.zip、.old、~ 等備份檔。

- **常見情境：**

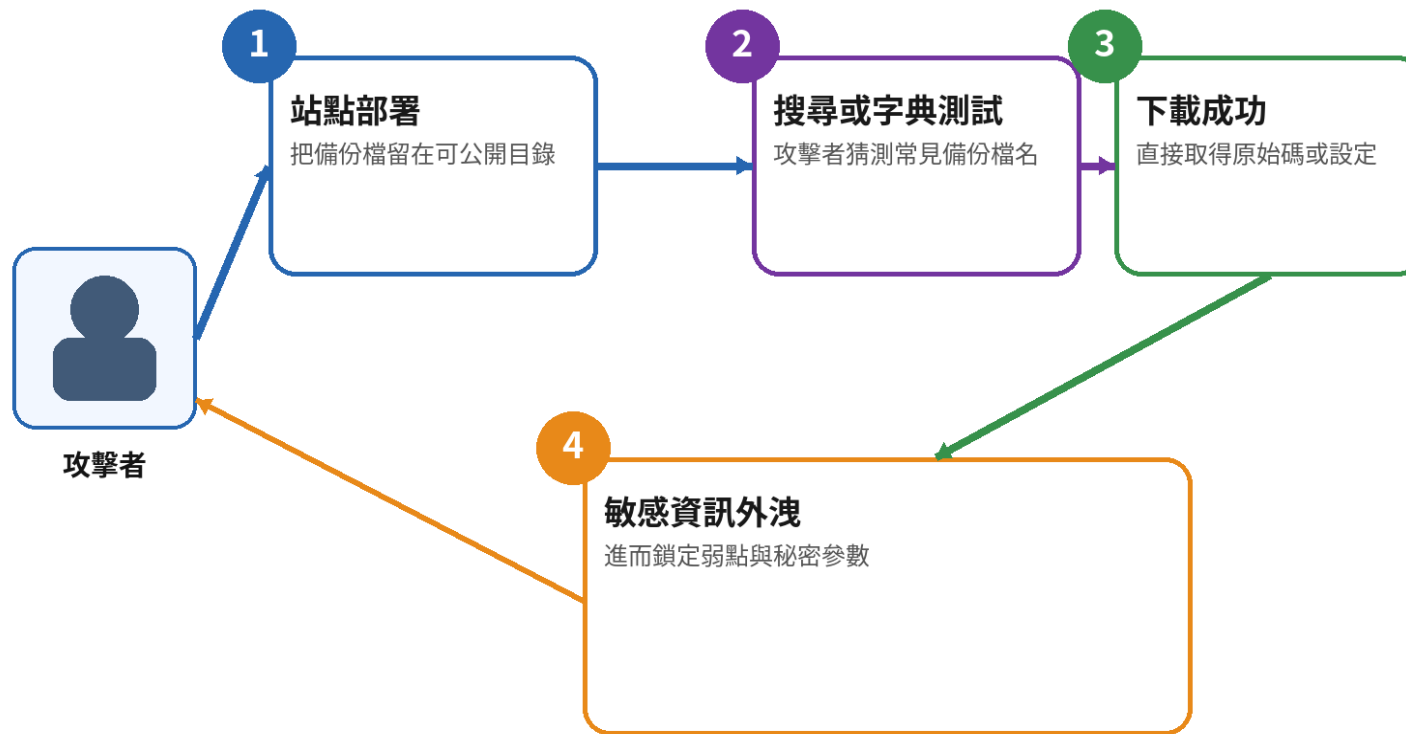
config.php.bak、index.php~、backup.zip、整站匯出檔。

- **可能影響：**

原始碼、帳密、**金鑰**與內部結構**直接外洩**。

- **防護重點：**

備份不得放在 Web 根目錄；部署前清理暫存、備份與舊版檔。



★ **重點：** 備份檔通常不是功能，卻可能一次**外洩**整個系統。

A03

Software Supply Chain Failures

風險不只來自過時元件，也包含套件來源、建置與更新流程

- 直接與間接相依套件都要納入清冊與風險評估
- 改善重點是可信任來源、版本鎖定、簽章驗證與最小權限
- DAST 只能看到部分前端元件，仍需搭配 SCA、SBOM 與版本管理

A03 | 供應鏈案例可從元件一路看到建置流程

舊版前端元件

jQuery 1.12.4、Bootstrap 4.0.0

已知元件弱點

PHPMailer、Log4Shell、Heartbleed 模擬案例

缺少 SRI

外部 CDN script / link 未驗證完整性

不可追溯來源

套件未鎖版、無 SBOM、來源不明

CI/CD 權限過大

建置權杖可修改正式環境

更新機制不足

下載後未驗證簽章或雜湊

A03 案例 | 掃到舊版 jQuery 只是供應鏈入口

實際情境

前端載入已知有弱點的舊版套件；但真正問題還包括來源、間接相依、更新責任與建置憑證。

如何驗證

ZAP 觀察前端版本；再以 SCA / SBOM 盤點完整相依關係與已知弱點。

安全改善

升級或移除元件、鎖定版本、限制來源、驗證完整性並建立更新時限。

修補證據

不只警示消失，還要能提出元件清冊、版本依據與更新紀錄。

過時元件 / 舊版 jQuery 風險 是什麼？

- **定義：**

使用已知有漏洞的前端或後端元件，等於把既有弱點帶進系統。

- **常見情境：**

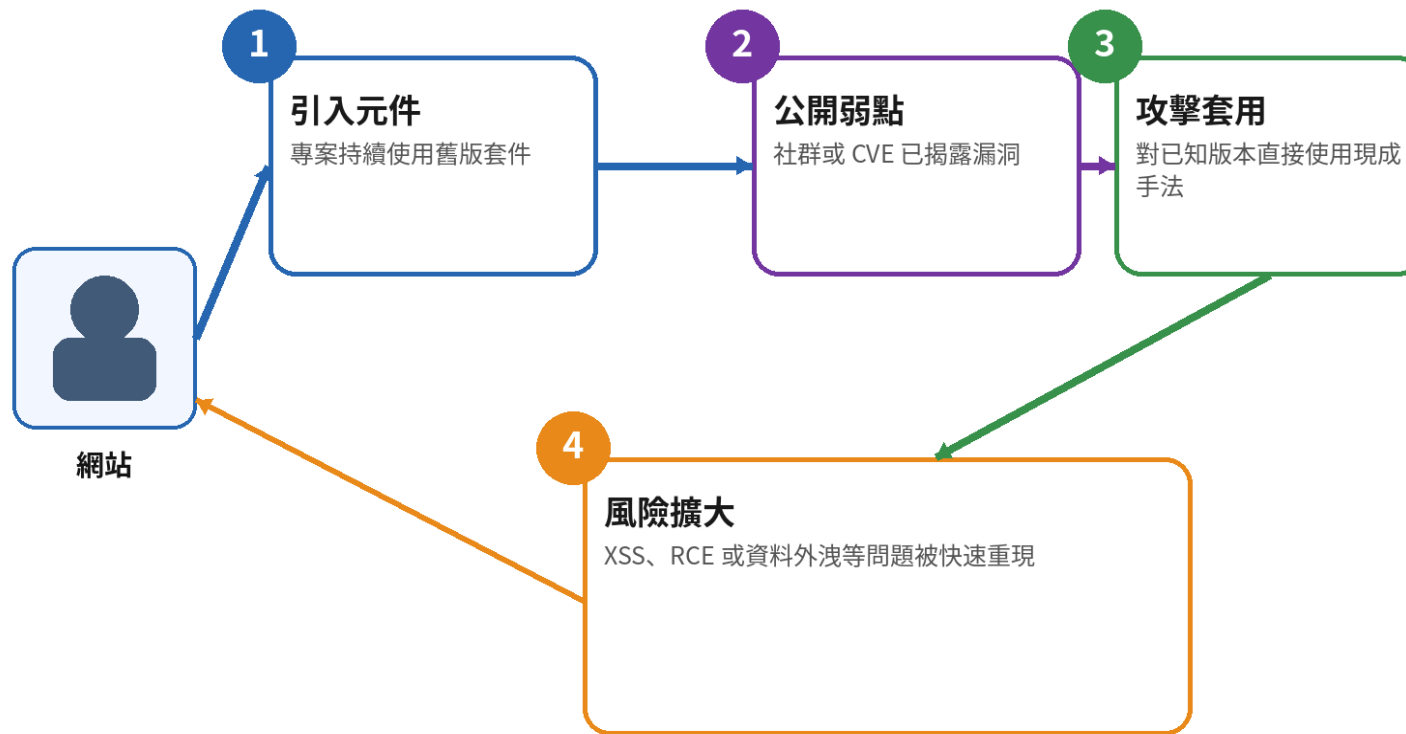
舊版 jQuery、未更新套件、棄用框架、過期外掛。

- **可能影響：**

已知攻擊手法可**直接**重現，降低攻擊門檻。

- **防護重點：**

建立 SBOM、追蹤 CVE、定期更新、替換無人維護元件。



★ **重點：** 掃到舊版元件只是入口，真正重點是後續修補與替換流程。

A04

Cryptographic Failures

可能是完全沒有加密、演算法太弱、金鑰外洩或使用方式錯誤

- 密碼、個資、權杖、Cookie、傳輸與備份需先做資料分類
- 採用成熟函式庫、現行標準與完整金鑰生命週期
- 固定金鑰、重複 IV、弱亂數與過時雜湊都會破壞保護效果

A04 | VulnCampus 的加密失效案例

明文密碼

資料庫直接儲存可讀取密碼

弱雜湊

MD5 或快速雜湊用於密碼

固定金鑰 / IV

所有環境與資料共用同一組值

寫死帳密

原始碼、前端或映像檔內含密碼與 API Key

Base64 當加密

編碼後仍可直接還原

弱亂數

可預測 Reset Token、Boundary 或識別碼

Cookie 明文

Remember Me 儲存帳密與角色

傳輸保護不足

HTTP、弱 TLS、憑證驗證或 HSTS 缺失

加密保護是一條完整生命週期，不是一個函式呼叫。



任一環節失效，都可能讓密文失去保護效果。

明文密碼 是什麼？

- 定義：

密碼以原文儲存、傳輸或可逆方式保存。

- 常見情境：

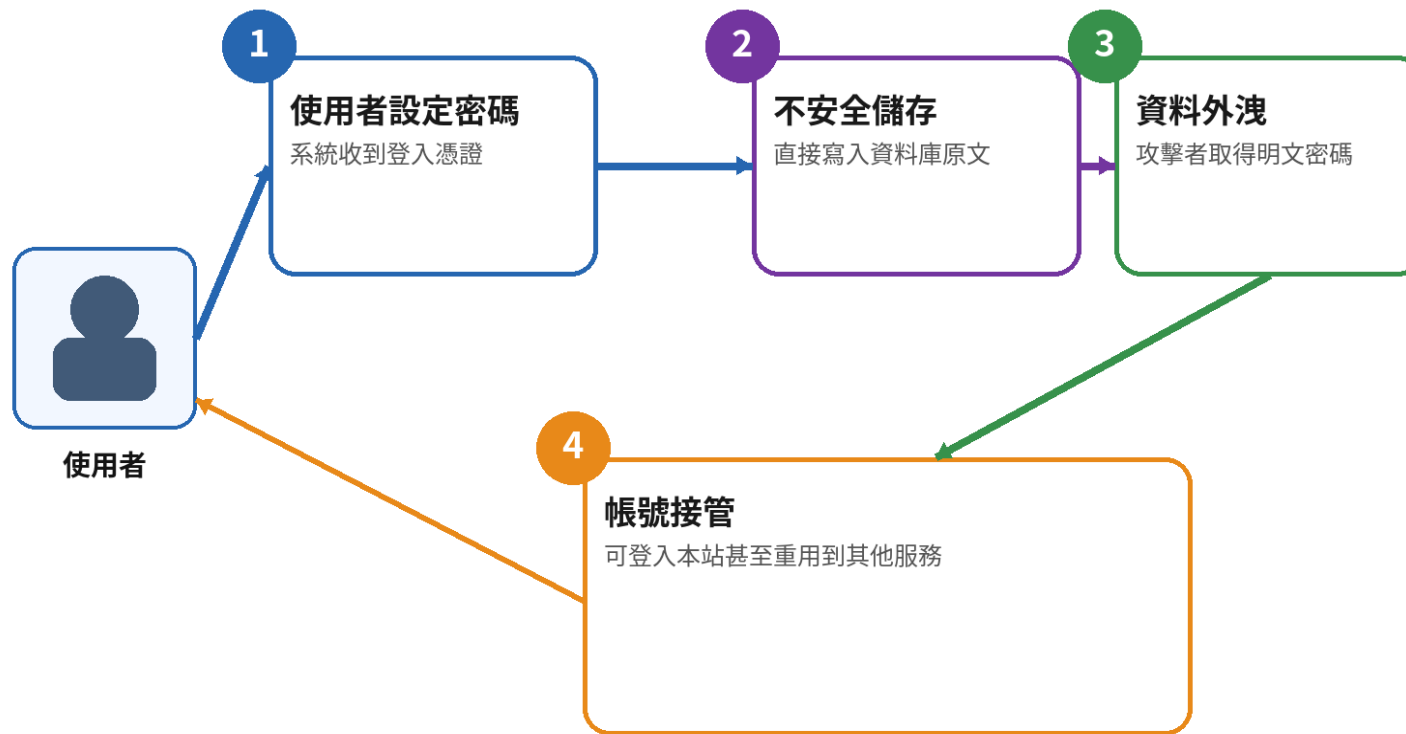
資料庫直接存密碼、匯出報表含密碼、寄送明文密碼。

- 可能影響：

資料庫一旦外洩，帳號立即失守且常擴散到其他站。

- 防護重點：

使用 bcrypt / Argon2 單向雜湊與適當 cost，絕不保存明文。



★ 重點：密碼不是一般資料，必須以單向雜湊保護。

使用 MD5 保護密碼 為何不夠？

- 定義：

MD5 速度過快且碰撞風險高，不適合保護密碼。

- 常見情境：

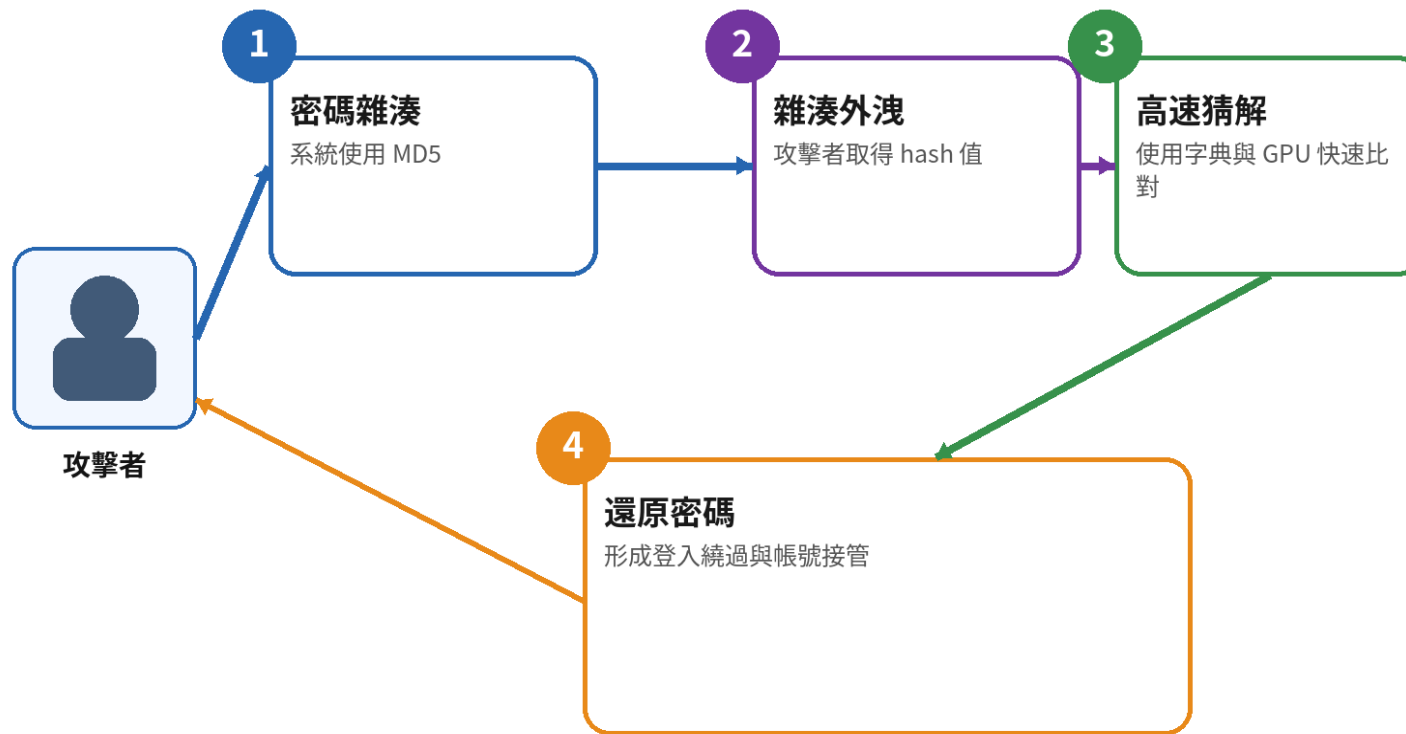
md5(password)、無 salt、舊系統沿用。

- 可能影響：

可被彩虹表或 GPU 快速破解。

- 防護重點：

改用 bcrypt、scrypt 或 Argon2，並規劃升級與重設密碼。



★ 重點：密碼保護重點不在「有沒有 hash」，而在「hash 是否抗暴力破解」。

A04 案例 | 寫死密碼與 API Key 會跟著程式一起外流

原始碼內寫死

```
$dbPassword = 'rootpassword';  
const API_KEY = 'AIza...';  
Git、備份檔、前端原始碼皆可取得
```

安全方向

```
$dbPassword = getenv('DB_PASSWORD');  
使用 Secrets Manager / 受控環境變數  
限制權限、記錄存取並定期輪替
```

安全修補應移除根因，並以相同輸入重新驗證。

寫死密碼 / API Key 外流 是什麼？

- **定義：**

把帳密、**API Key**、憑證寫在程式或前端檔案中。

- **常見情境：**

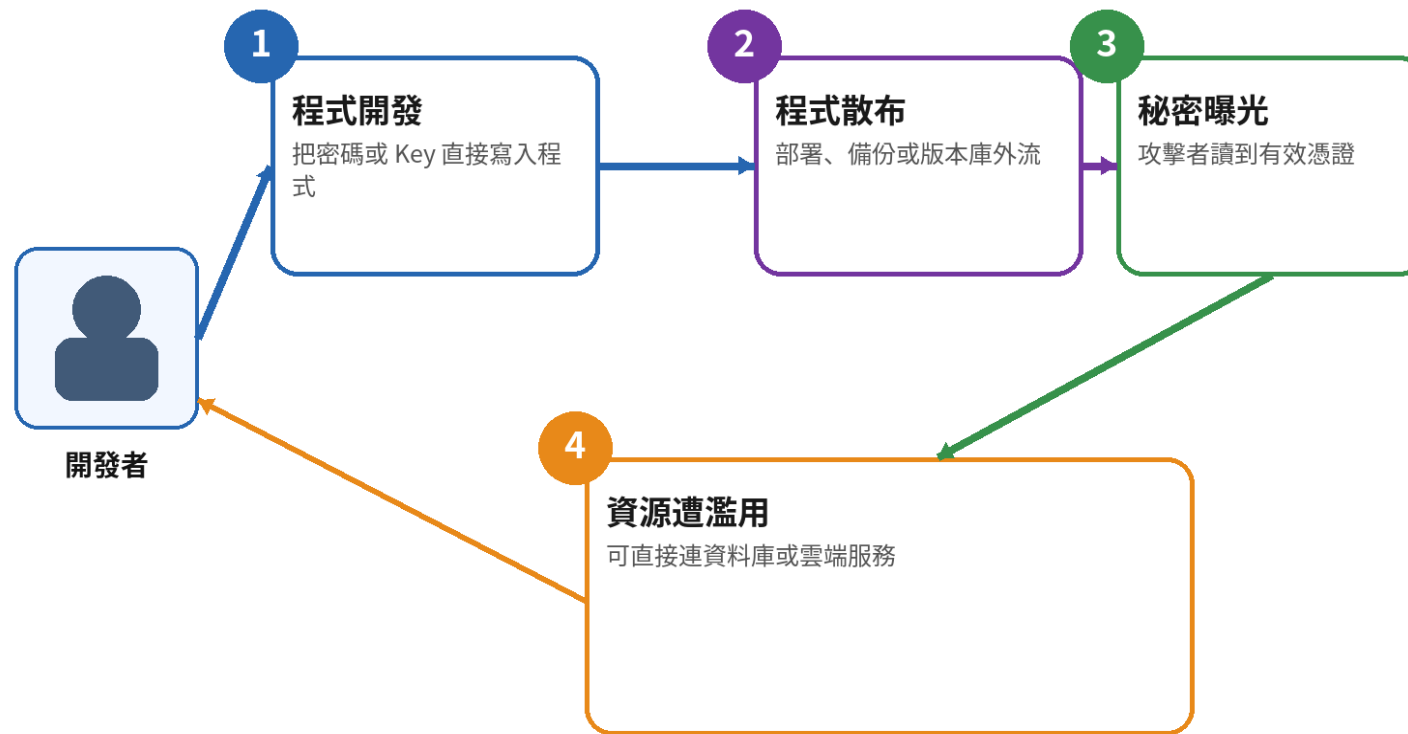
硬編碼 DB **密碼**、雲端**金鑰**、第三方服務 **token**。

- **可能影響：**

程式碼**外洩**或前端檢視時，秘密資料會一起曝光。

- **防護重點：**

使用秘密管理服務、環境變數、**權限**分離與定期輪替。



★ **重點：** 秘密不應跟著程式一起散布，應獨立管理與輪替。

A04 案例 | 固定金鑰與固定 IV 讓不同資料產生可比對模式

固定值重複使用

```
$key = '0123456789abcdef';  
$iv  = '0000000000000000';  
不同使用者、不同環境共用同一組值
```

安全方向

金鑰由 CSPRNG 產生並置於受控金鑰服務
每次加密使用符合模式要求的新 Nonce / IV
優先採用 AES-GCM 等驗證式加密

安全修補應移除根因，並以相同輸入重新驗證。

固定金鑰 / 固定 IV 風險 是什麼？

- **定義：**

所有資料都使用同一把**金鑰**或固定 **IV**，加密模式會暴露規律。

- **常見情境：**

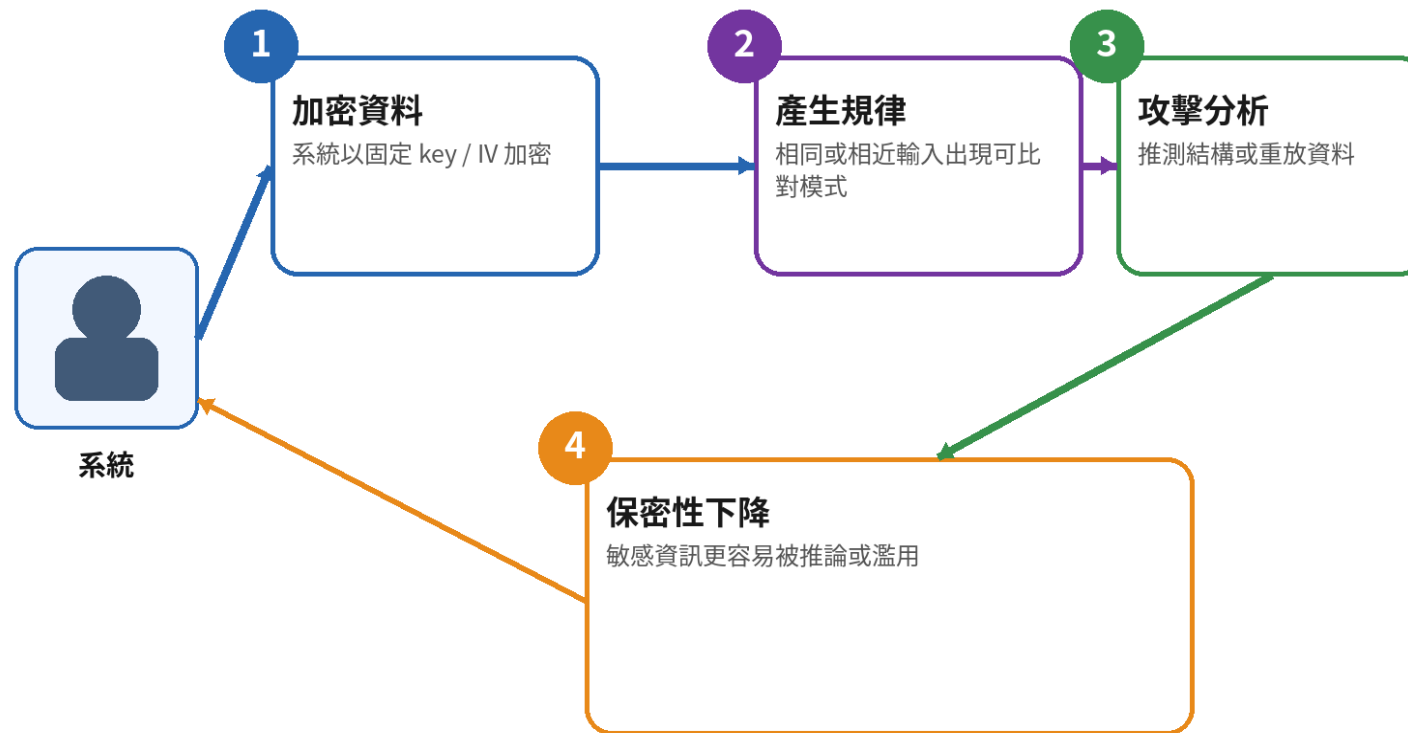
程式寫死 AES key、CBC 固定 **IV**、多人共用同一 secret。

- **可能影響：**

不同資料產生可比對模式，降低保密性。

- **防護重點：**

安全產生 **IV** / nonce、**金鑰**分離、輪替與妥善保存。



★ **重點：** 演算法正確不代表實作安全，key 與 **IV** 的管理同樣重要。

A04 案例 | Base64 只是編碼，Cookie 也不是祕密保管箱

看似不可讀

```
$token = base64_encode($nationalId);  
setcookie('remember_pwd', $password);  
任何取得者都能還原或竄改
```

安全方向

不需要的敏感資料不要保存
Cookie 僅保存隨機代用權杖
敏感狀態由伺服器端管理並設定安全屬性

安全修補應移除根因，並以相同輸入重新驗證。

Base64 不是加密 是什麼意思？

- 定義：

Base64 只是編碼，不提供保密性。

- 常見情境：

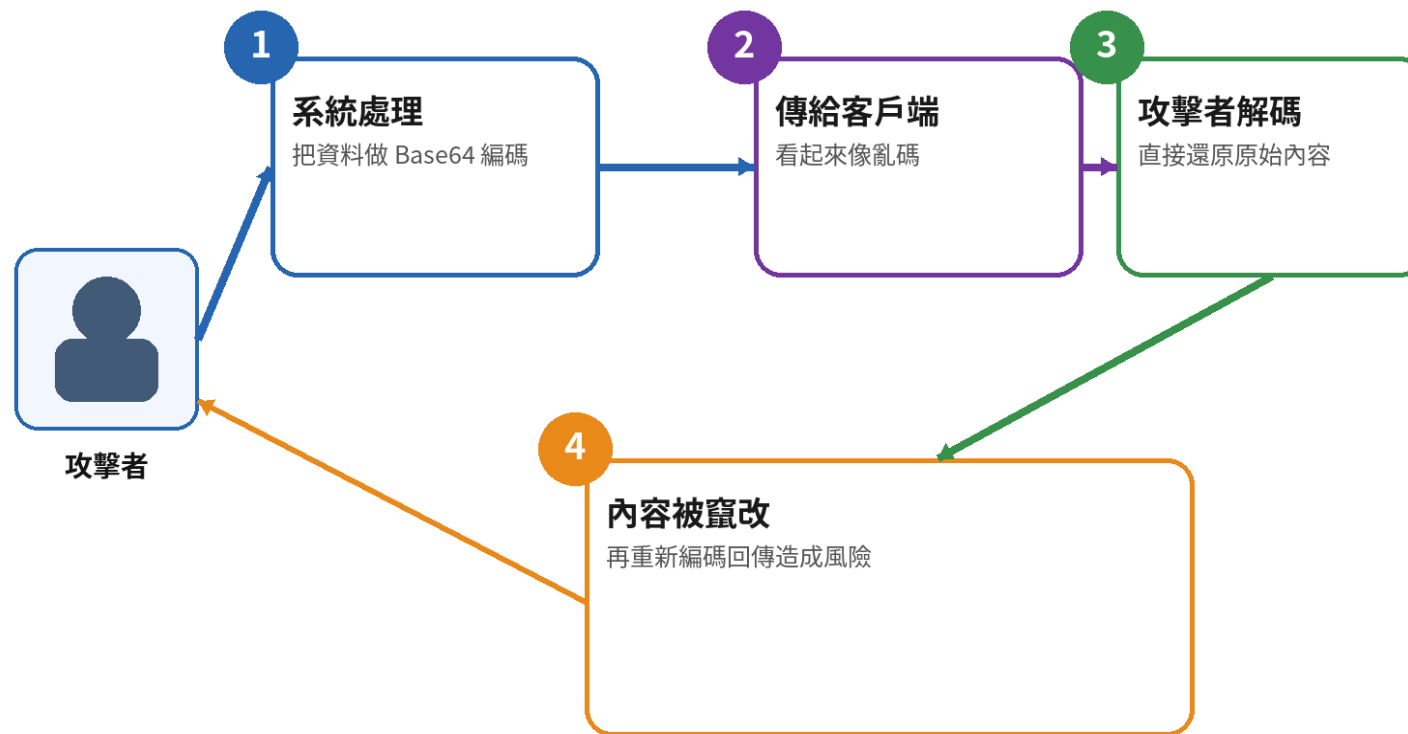
把 Cookie、Token 或設定值做 Base64 後誤以為安全。

- 可能影響：

攻擊者可輕易還原內容，再進一步竄改。

- 防護重點：

若需保密請用正確加密；若需完整性請加入簽章驗證。



★ 重點：看起來像亂碼，不代表資料真的受到保護。

A04 案例 | 可預測亂數會讓權杖與 Boundary 被推算

攻擊資料流



改用 `random_bytes()` 等 CSPRNG，並加入時效、綁定身分與一次性使用。

A04 | 金鑰生命週期比「把資料加密」更重要

- **產生：**

使用足夠熵的 CSPRNG，不自行設計金鑰格式

- **儲存：**

與資料分離，限制存取並避免提交至版本庫

- **使用：**

定義用途、環境與權限，避免同一金鑰跨系統重複使用

- **輪替：**

設定有效期、版本與平順換鑰機制

- **撤銷：**

外洩時能停用、重新加密並追蹤受影響範圍

金鑰生命週期管理 是什麼？

- 定義：

金鑰不只要存在，還要能建立、保護、輪替、註銷與追蹤。

- 常見情境：

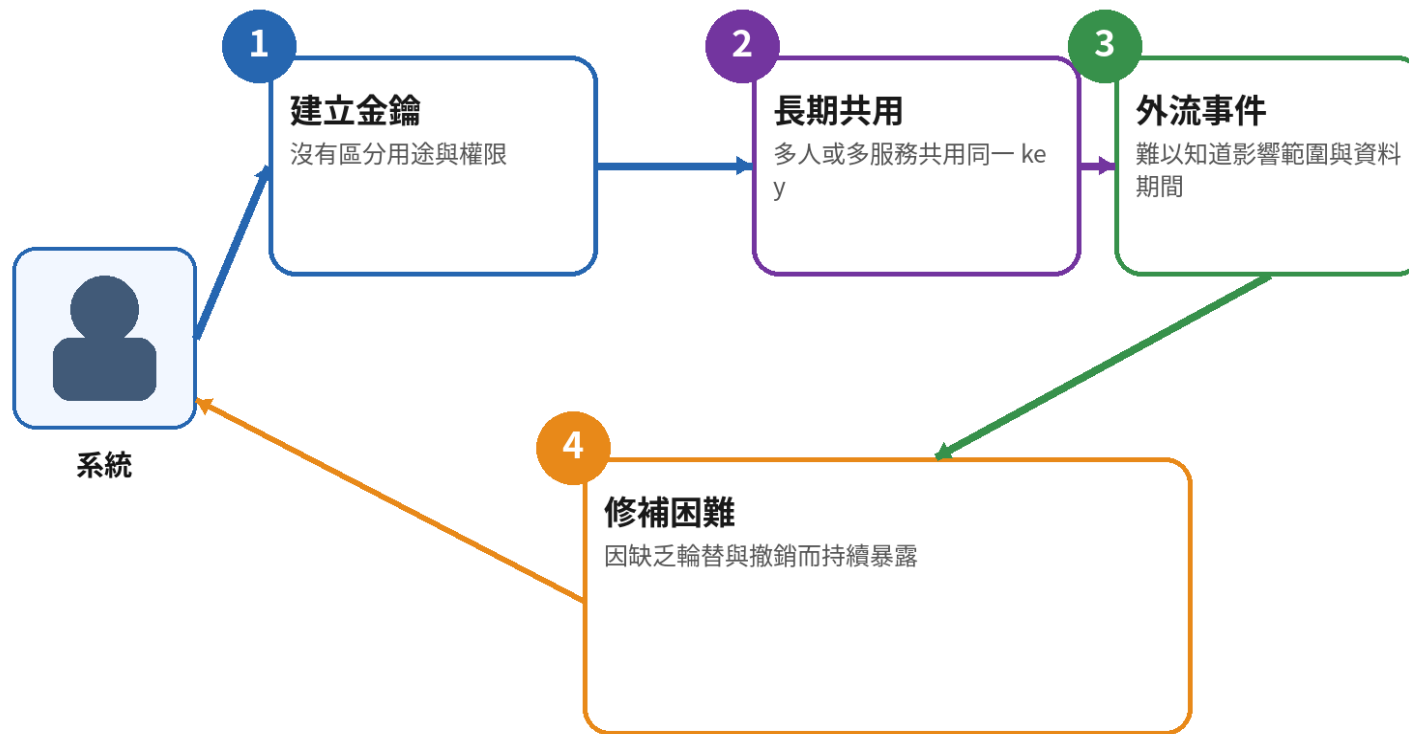
永久不換 key、多人共用、無版本、離職後仍可用。

- 可能影響：

一旦外流，長期資料都可能受影響且難以止血。

- 防護重點：

最小可見性、分權保管、版本化輪替與撤銷機制。



★ 重點：加密不是一次性工作，金鑰治理才是長期安全的核心。

A05

Injection

不可信任資料進入瀏覽器、資料庫、命令列等解譯器

- 解譯器把輸入的一部分當成指令或語法執行
- 核心防護是讓資料與指令分離，再搭配情境化編碼與伺服器端驗證
- OWASP 2025 將 SQL、XSS、NoSQL、OS Command、ORM、LDAP、EL 等納入

攻擊資料流



輸入來源可包含 URL、表單、JSON、Cookie、標頭、XML 與檔案內容。

A05 | VulnCampus 已涵蓋的注入類型

類型	說明	案例
SQL Injection	login、courses、sqli_variants、User-Agent 標頭	資料庫
XSS	Reflected、Stored、DOM-based	瀏覽器 / DOM
Command Injection	admin/ping、command_injection	作業系統命令列
CSS Injection	css_injection.php	瀏覽器 CSS 解析器
CRLF Injection	crlf_injection.php	HTTP 標頭
Eval Code Injection	eval_injection.php	PHP 執行環境
XPath Injection	xpath_injection.php	XML / XPath
LFI / RFI	file_inclusion.php	PHP include / require

SQL Injection 原理 | 輸入突破資料邊界，改寫 WHERE 條件

攻擊資料流



影響可能包含登入繞過、資料外洩、竄改、刪除，甚至資料庫或主機控制。

SQL Injection 是什麼？

• 定義：

當使用者輸入被**直接**拼進 **SQL** 指令，資料就可能**改變**查詢結構。

• 常見攻擊：

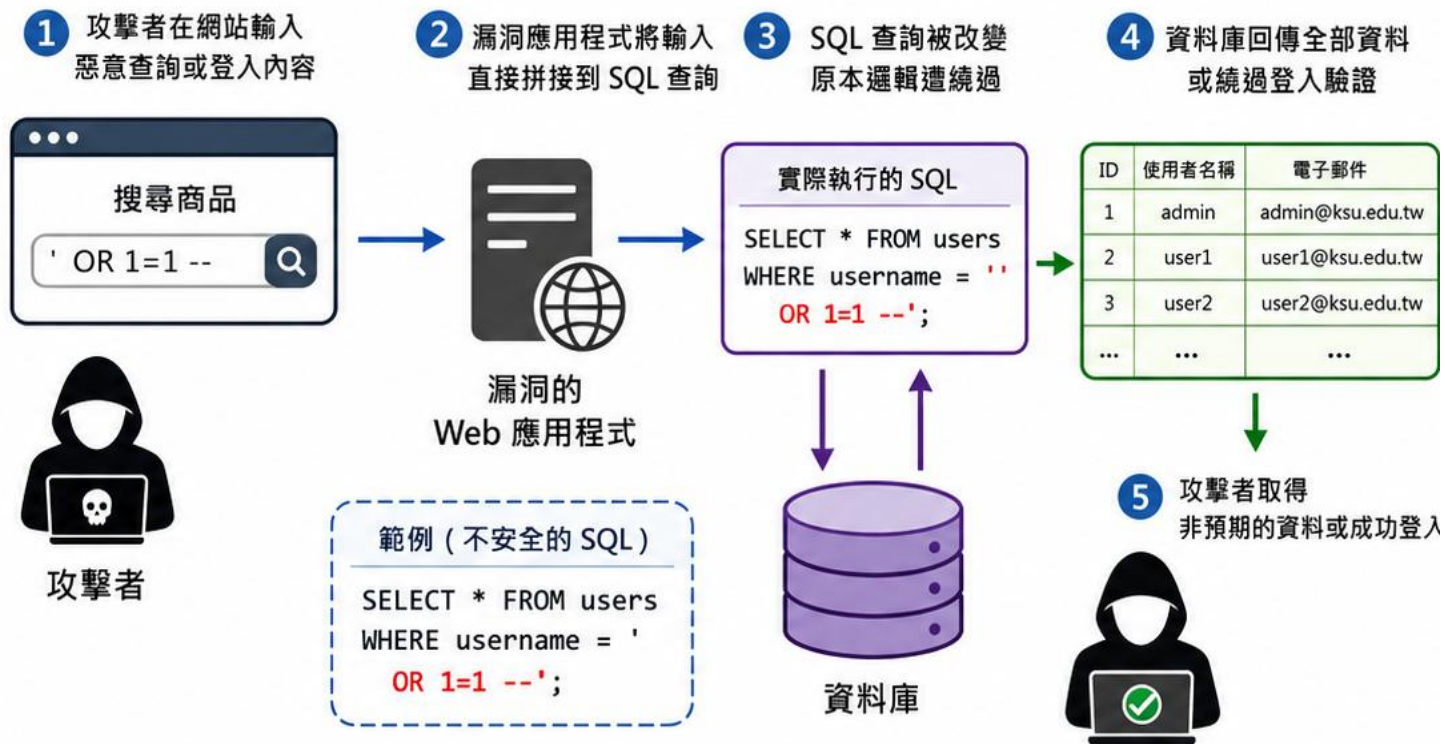
' **OR 1=1 --**、**UNION SELECT**、錯誤訊息探測、時間延遲測試。

• 可能影響：

登入**繞過**、資料**外洩**、竄改或刪除資料，嚴重時甚至**控制資料庫**。

• 防護重點：

參數化查詢、**預備語句**、**輸入驗證**、**最小資料庫權限**、**不要回傳詳細錯誤**。



★ **重點：**SQLi 的關鍵不是「有特殊字元」，而是「輸入能不能**改變**原本 **SQL** 的結構」。

SQL Injection 常見類型與辨識方式

類型	主要特徵	觀察證據	VulnCampus
UNION-based	合併額外 SELECT 結果	頁面直接回顯其他資料表內容	sql_i_variants?tab=union
Error-based	利用錯誤訊息帶出結構或資料	SQL 錯誤、資料庫版本、欄位資訊	courses、course_detail
Boolean-based Blind	以 True / False 差異逐位猜測	頁面內容或狀態有可判別差異	sql_i_variants?tab=boolean
Time-based Blind	條件成立時刻意延遲	回應時間穩定增加	sql_i_variants?tab=time
Stacked Queries	以分號追加第二段指令	資料被修改或刪除	視資料庫與驅動支援
Stored Procedure	程序內仍動態拼接 SQL	CALL / EXEC 參數改變查詢	sql_i_variants?tab=sp
Second-order	惡意值先儲存，之後才被拼接執行	首次輸入正常、後續流程觸發	可作延伸練習
Header / Cookie SQLi	非表單欄位進入 SQL	User-Agent、Referer、Cookie 觸發	login 日誌寫入

SQLi | UNION-based : 有回顯時快速合併敏感資料

實際情境

title 參數直接拼接；攻擊值以 UNION SELECT 讓另一個查詢結果出現在原頁面。

如何驗證

先確認欄位數量與型別，再於授權 Lab 觀察是否回顯 username、password_hash 或 role。

安全改善

使用 PDO Prepared Statement；資料庫帳號採最小權限，避免可讀取不必要資料表。

修補證據

相同輸入修補後只作為搜尋文字，不再改變欄位數量與結果集合。

UNION-based SQLi 是什麼？

- **定義：**

利用 UNION 合併查詢，把原本不會回顯的資料一起顯示。

- **常見情境：**

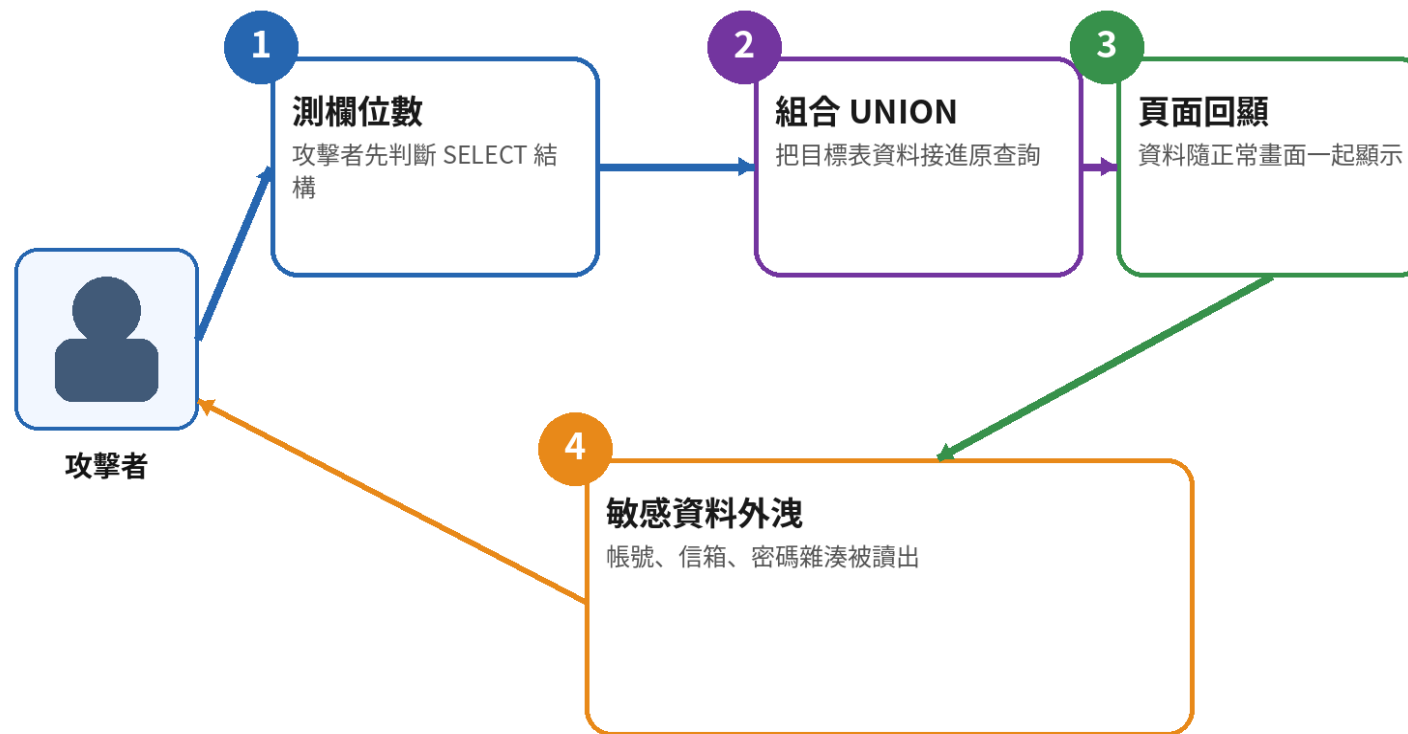
先測試欄位數，再以 **UNION SELECT** 拉出 users、orders 等資料。

- **可能影響：**

有回顯頁面時可快速**外洩敏感**資料。

- **防護重點：**

參數化查詢、**最小權限**與錯誤抑制；**不要**把輸入拼進 **SQL**。



★ **重點：** 有回顯時，UNION-based SQLi 的資料**外洩**速度通常很快。

SQLi | Error-based : 錯誤訊息本身就是情報

實際情境

單引號或型別錯誤造成 SQL Exception，頁面直接顯示查詢、資料庫驅動與實體路徑。

如何驗證

比較正常值、單引號、陣列型參數與超長輸入的狀態碼、內容及回應長度。

安全改善

參數化查詢並集中例外處理；對外只顯示一般訊息，細節寫入受控日誌。

修補證據

修補後回應不含 SQL、Stack Trace、資料庫型別或檔案路徑。

Error-based SQLi 是什麼？

- **定義：**

故意觸發**資料庫**錯誤，讓錯誤訊息替**攻擊者**洩漏情報。

- **常見情境：**

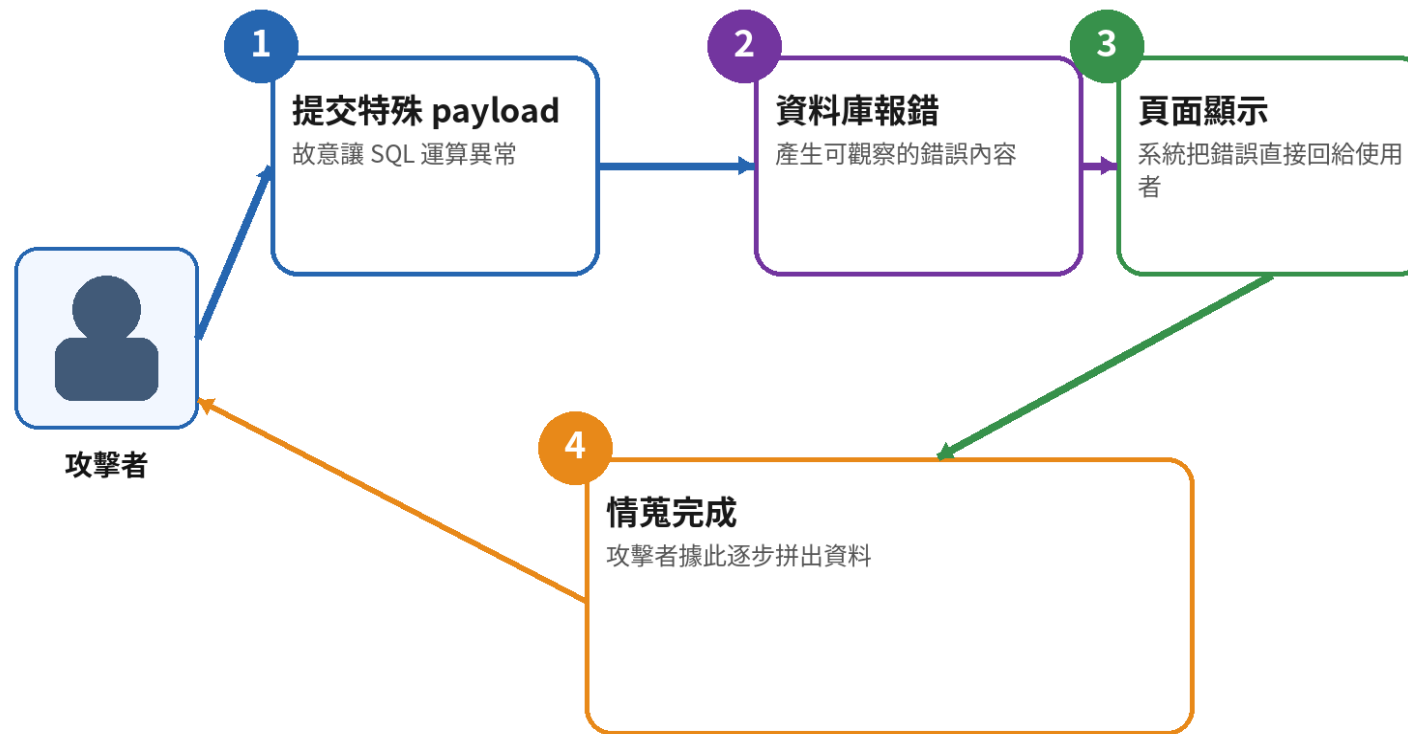
型別轉換錯誤、XML 函式錯誤、group by / extractvalue 等。

- **可能影響：**

SQL 結構、資料片段、版本與**資料庫**類型被辨識。

- **防護重點：**

關閉**詳細錯誤**輸出、**參數化**查詢、集中記錄內部錯誤。



★ **重點：** 錯誤訊息不是雜音，而可能是**直接的資料外洩**通道。

SQLi | Boolean-based Blind：沒有資料回顯，也能逐步猜解

攻擊資料流



判斷依據可能是文字、狀態碼、內容長度或其他穩定差異。

Boolean-based Blind SQLi 是什麼？

- **定義：**

即使沒有資料回顯，也可利用真 / 假差異逐步猜解內容。

- **常見情境：**

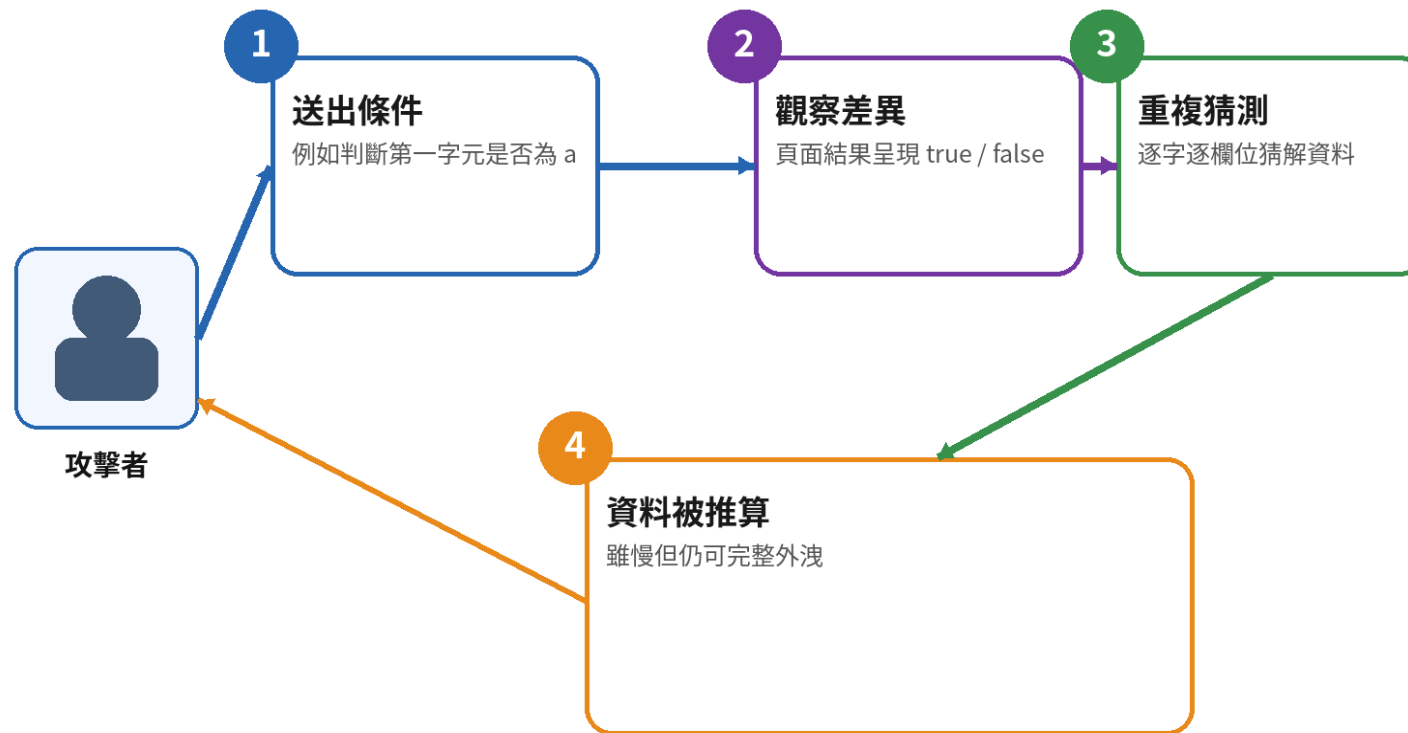
觀察頁面是否顯示「有資料 / 無資料」或回應碼差異。

- **可能影響：**

可慢慢猜出帳號、表名、密碼雜湊等。

- **防護重點：**

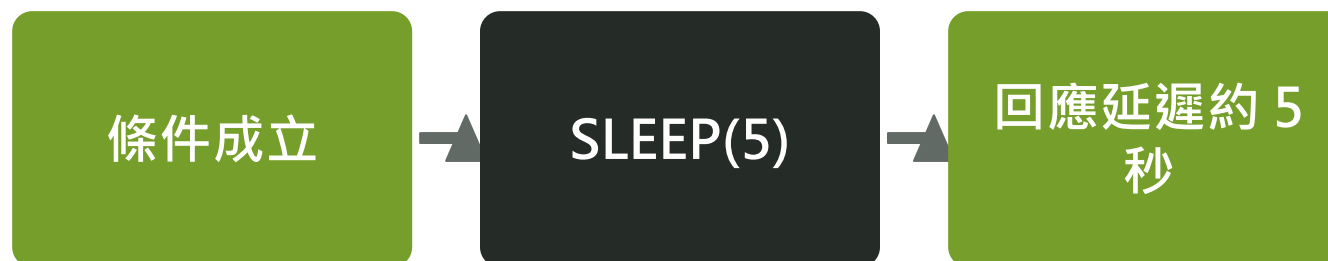
參數化查詢、統一回應、避免條件被輸入改寫。



★ **重點：** 沒有回顯不代表安全，回應差異本身就能成為訊號。

SQLi | Time-based Blind : 用回應延遲當作 True / False

攻擊資料流



測試時需建立基準並多次比較，避免把網路抖動誤判為弱點。

Time-based Blind SQLi 是什麼？

- **定義：**

利用延遲函式讓回應時間成為 True / False 的判斷訊號。

- **常見情境：**

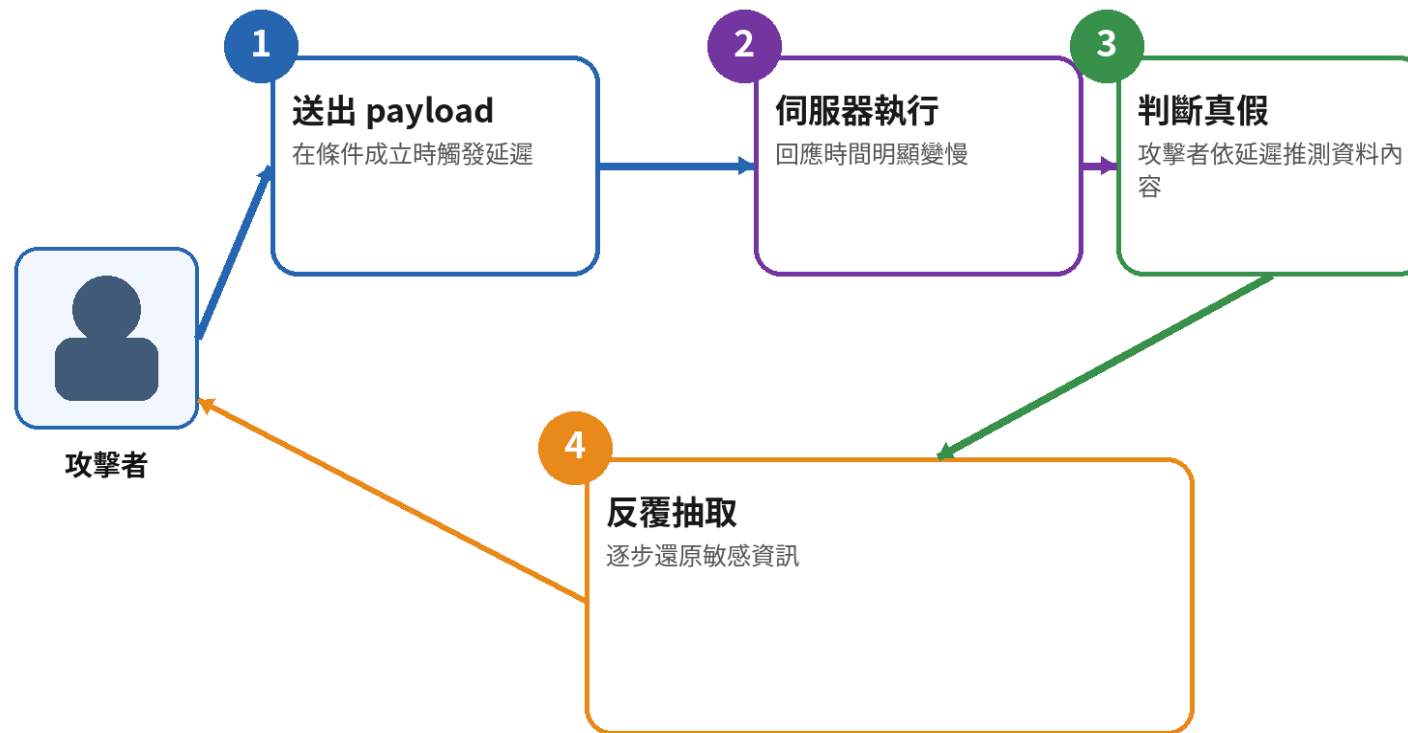
sleep(5)、benchmark()、pg_sleep() 等時間延遲函式。

- **可能影響：**

在完全無回顯時仍可逐步抽出資料。

- **防護重點：**

參數化查詢、限制危險函式、監控異常慢查詢。



★ **重點：** 只要輸入能控制 SQL 結構，連回應時間都可能變成外洩通道。

SQLi | Stored Procedure 與 HTTP Header 也可能被注入

實際情境

Stored Procedure 內動態組字串仍不安全；User-Agent 等標頭若直接拼進日誌 SQL，一樣可改變查詢。

如何驗證

檢查 CALL / EXEC 實作、所有標頭與 Cookie 的資料流，不只測試畫面上的輸入框。

安全改善

程序內外都使用參數綁定；將日誌欄位當成不可信任資料處理。

修補證據

掃描與程式碼檢查均確認沒有動態 SQL 字串拼接。

Stored Procedure / HTTP Header 注入 是什麼？

- 定義：

注入點不只在表單，也可能出現在 Header、Cookie 或預存程序參數。

- 常見情境：

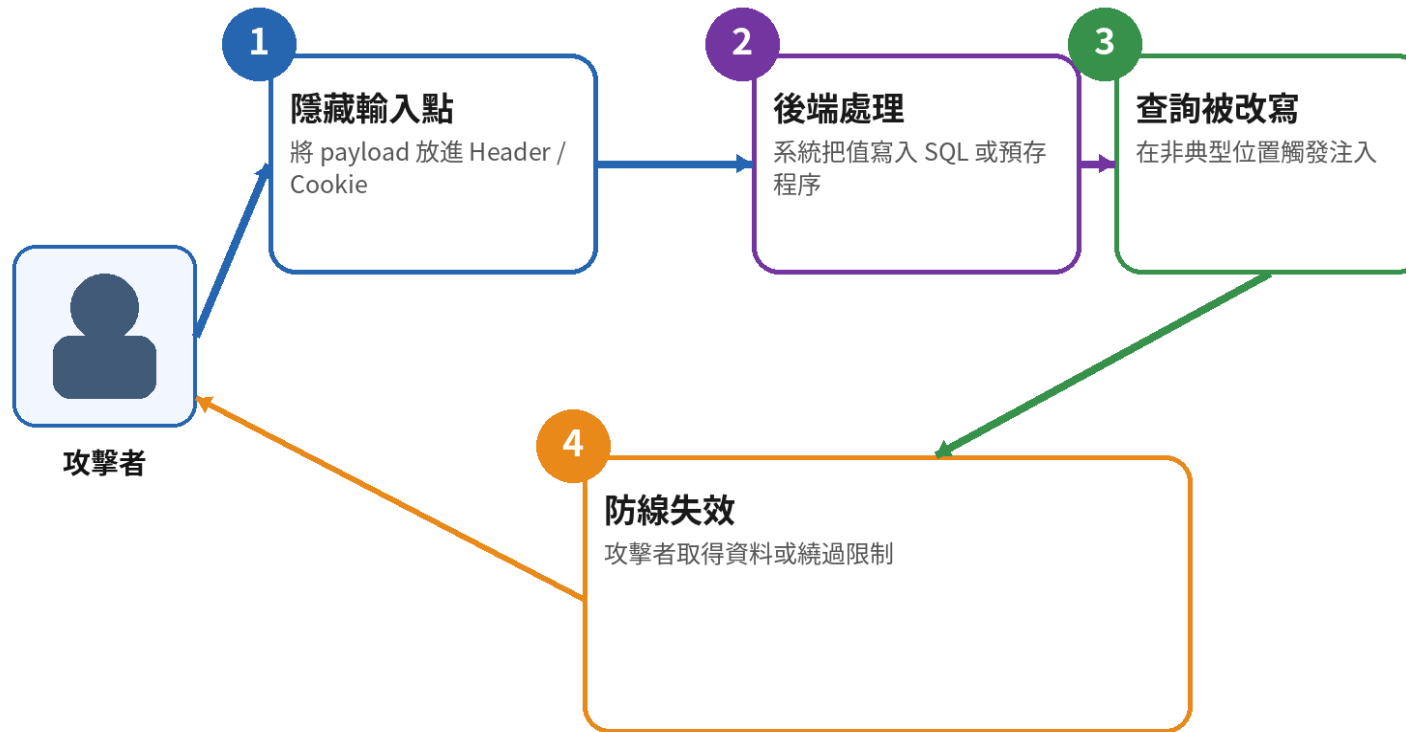
User-Agent、X-Forwarded-For、Referer 被寫入 SQL。

- 可能影響：

繞過既有檢測，從較少注意的位置觸發注入。

- 防護重點：

所有輸入一律視為不可信；預存程序也需安全綁定參數。



★ 重點：注入的本質是資料改變指令結構，而不是「一定來自某個欄位」。

SQL Injection 修補 | 參數化查詢讓資料不再改變 SQL 結構

危險：字串拼接

```
$q = $_GET['q'];  
$sql = "SELECT * FROM courses WHERE  
title LIKE '%$q%'";  
$rows = $pdo->query($sql);
```

安全：Prepared Statement

```
$stmt = $pdo->prepare('SELECT * FROM  
courses WHERE title LIKE :q');  
$stmt->execute(['q' => '%' . $q .  
'%']);  
$rows = $stmt->fetchAll();
```

安全修補應移除根因，並以相同輸入重新驗證。

XSS 原理 | 伺服器信任輸入，瀏覽器便替攻擊者執行

攻擊資料流



XSS 的解譯器是瀏覽器；修補必須依 HTML、屬性、URL、JavaScript 等輸出情境處理。

XSS (跨站腳本) 是什麼？

• 定義：

網站把未妥善處理的輸入**直接**輸出到頁面，讓瀏覽器替**攻擊者**執行**惡意**程式碼。

• 常見型態：

Reflected、Stored、**DOM**-based。

• 可能影響：

竊取 **Cookie** / **Session**、冒用**帳號**、釣魚畫面、改寫頁面內容、植入**惡意**連結。

• 防護重點：

依輸出情境編碼、避免危險 **DOM** API、使用 **CSP**、框架預設 escaping。



★ **重點：** XSS 的本質是「瀏覽器把**攻擊者**的資料當成程式來執行」。

XSS 三種主要型態

型態	Payload 經過哪裡	誰會受影響	VulnCampus
Reflected XSS	請求進入伺服器後立即反射回應	點擊惡意連結的使用者	xss_reflected.php
Stored XSS	先儲存在資料庫，再提供給其他使用者	所有瀏覽該內容的使用者	xss_stored.php
DOM-based XSS	前端 JavaScript 從 Source 寫入危險 Sink	執行該前端流程的使用者	xss_dom.php、checkin.php

XSS | Reflected : Payload 跟著連結進入並立即反射

實際情境

keyword 參數未編碼就放入 HTML，惡意連結使瀏覽器執行 Script。

如何驗證

在授權環境觀察 Alert 的 Parameter、Attack、Evidence 與回應中未編碼內容。

安全改善

依 HTML 情境使用 htmlspecialchars(..., ENT_QUOTES, 'UTF-8')，並補充 CSP 防禦縱深。

修補證據

修補後 Payload 以純文字顯示，不產生標籤、事件屬性或 Script 執行。

Reflected XSS 是什麼？

- 定義：

惡意腳本跟著連結或請求送進網站，立即被反射回受害者頁面。

- 常見情境：

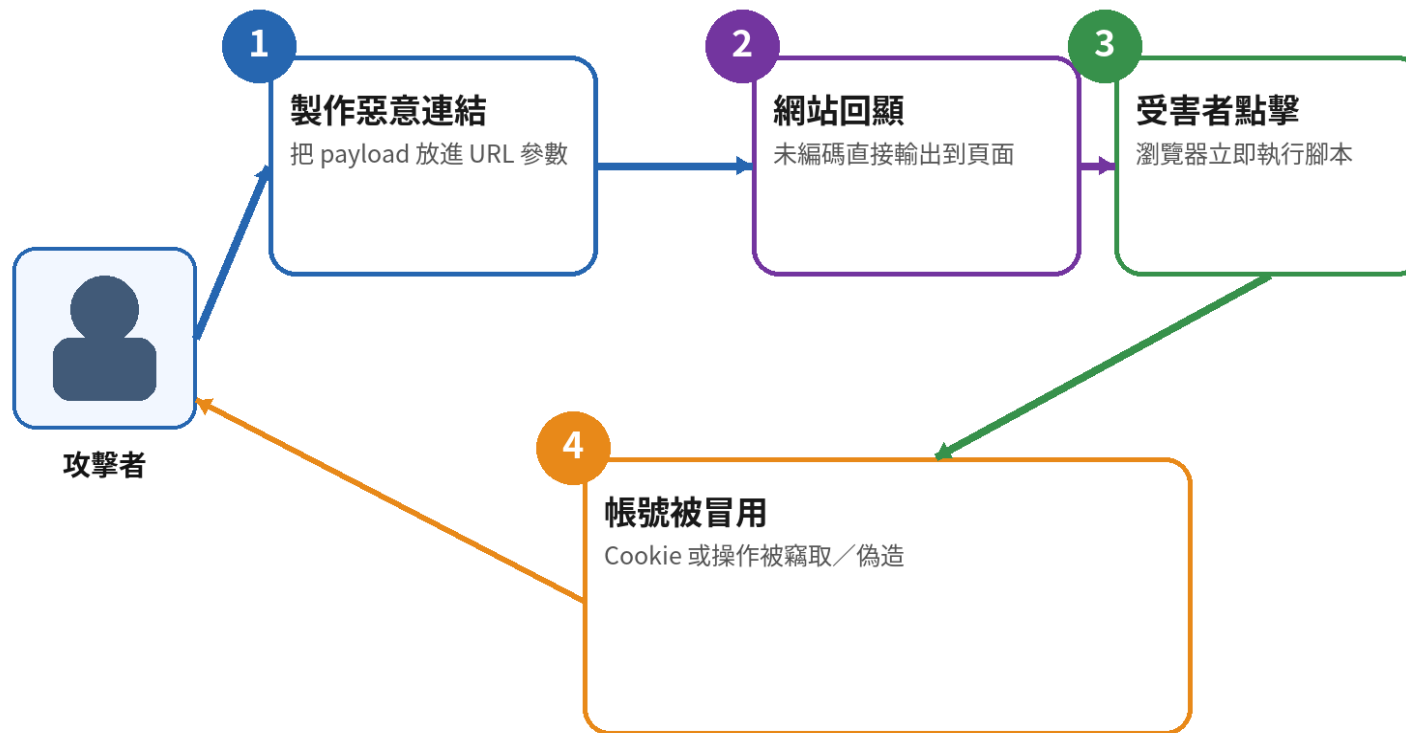
搜尋、錯誤頁、回顯參數頁面。

- 可能影響：

受害者點擊一次惡意連結就可能執行腳本。

- 防護重點：

輸出編碼、限制危險 DOM API、對輸入做情境化處理。



★ 重點：Reflected XSS 通常需要受害者觸發，但一點擊就可能中招。

XSS | Stored：一次寫入，後續每位瀏覽者都可能受害

實際情境

留言內容原樣存入資料庫並輸出；管理者查看留言時也會執行攻擊碼。

如何驗證

建立測試留言、重新開啟頁面並以不同帳號確認影響範圍。

安全改善

資料庫使用參數化查詢；輸出時依情境編碼；Session Cookie 設定 HttpOnly。

修補證據

修補後歷史資料與新留言皆以文字呈現，Cookie 亦不可由 JavaScript 讀取。

Stored XSS 是什麼？

- 定義：

惡意腳本先被存進資料庫，之後每位瀏覽者都可能受害。

- 常見情境：

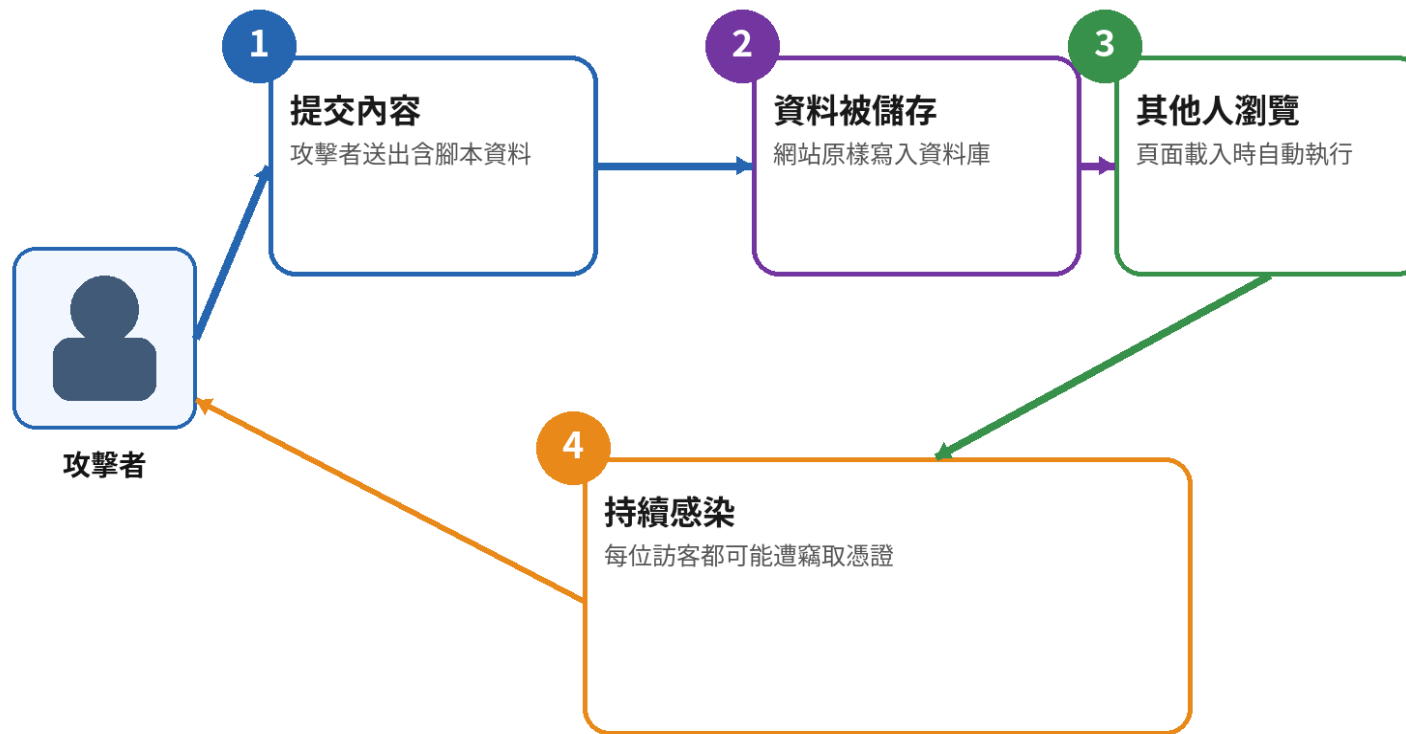
留言板、暱稱、評論、公告內容。

- 可能影響：

一次植入，多人受害，影響範圍通常比 Reflected 更大。

- 防護重點：

輸入淨化 + 輸出編碼，管理後台也不能信任資料。



★ 重點：Stored XSS 的危險在於「一次寫入、持續觸發」。

XSS | DOM-based : 漏洞完全可能發生在前端

實際情境

JavaScript 讀取 `location.hash` 或 API JSON，直接指定給 `innerHTML`。

如何驗證

追蹤 Source 到 Sink : `location`、`postMessage`、JSON → `innerHTML`、`document.write`、`eval`。

安全改善

優先使用 `textContent` / 安全 DOM API ; 必要時採經驗證的 HTML Sanitizer。

修補證據

修補後 Hash 或 JSON 中的標籤只以文字呈現，不進入危險 Sink。

DOM-based XSS 是什麼？

- 定義：

漏洞發生在前端 JavaScript，資料未經安全處理直接進入 DOM。

- 常見情境：

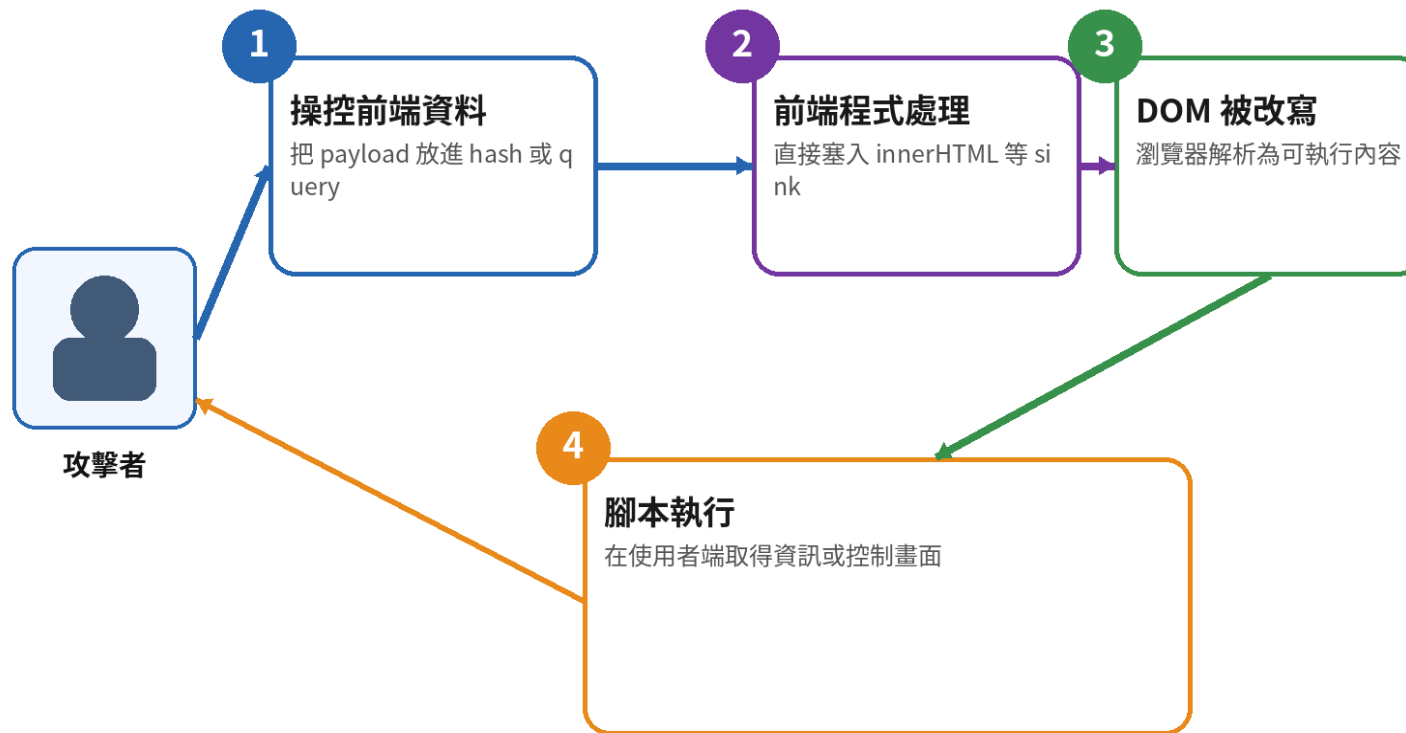
location.hash、document.write、innerHTML、URL 參數處理。

- 可能影響：

伺服器端可能看不到異常，但瀏覽器端仍會執行腳本。

- 防護重點：

避免危險 sink、使用安全 API、框架預設 escaping。



★ **重點：** DOM XSS 可能完全發生在前端，因此測試不能只看後端回應。

Command Injection | 輸入被接到 Shell，影響可直接到主機

攻擊資料流



優先避免呼叫 Shell；僅以 `escapeshellarg()` 補救並不足以取代允許清單與安全 API。

Command Injection (命令注入) 是什麼？

- 定義：

當未信任輸入進入 **Shell** 或系統命令字串，**攻擊者**就可能追加自己的命令。

- 常見攻擊：

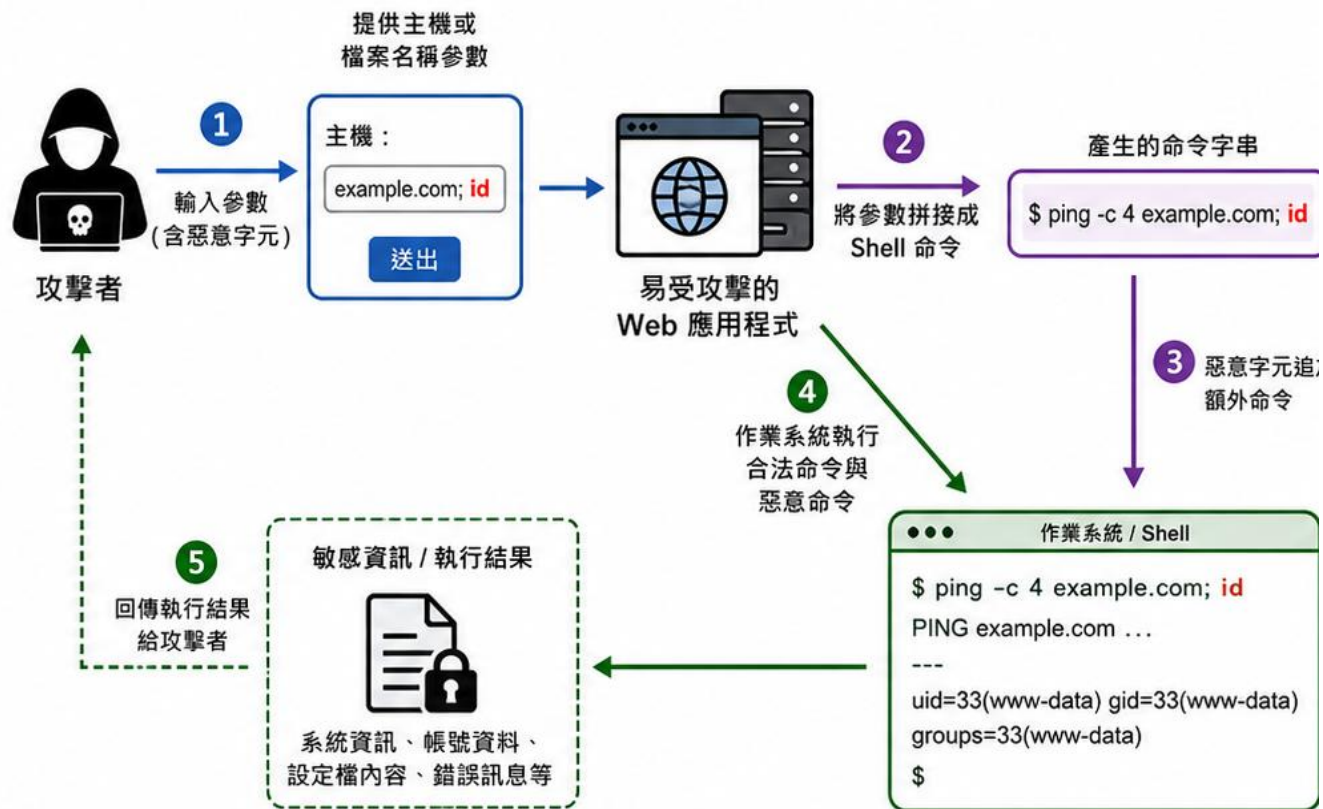
; id、**&&** whoami、**|** cat /etc/passwd
、反引號與命令替換。

- 可能影響：

讀取主機資訊、執行任意命令、下載**惡意**程式、橫向移動、完全接管伺服器。

- 防護重點：

避免呼叫 **Shell**、改用函式庫 API、固定命令模板、嚴格**允許清單**與**最小權限**。



★ **重點：** 命令注入的關鍵在於「輸入變成了命令的一部分」。

CSS Injection | 沒有 JavaScript，也可能追蹤、遮蔽或外洩資料

實際情境

使用者可控制 style 或 CSS 規則，載入外部資源、遮蔽按鈕或以選擇器推測頁面資訊。

如何驗證

檢查 style、class、CSS 變數、@import、url() 與可控屬性值。

安全改善

避免接受任意 CSS；以有限選項映射伺服器端樣式，並套用 CSP。

修補證據

修補後輸入只能選擇核准的色彩 / 主題，不得產生任意 CSS 語法。

CSS Injection 是什麼？

- **定義：**

未妥善處理的輸入進入 style 或 CSS，
雖非 JavaScript 也可能造成風險。

- **常見情境：**

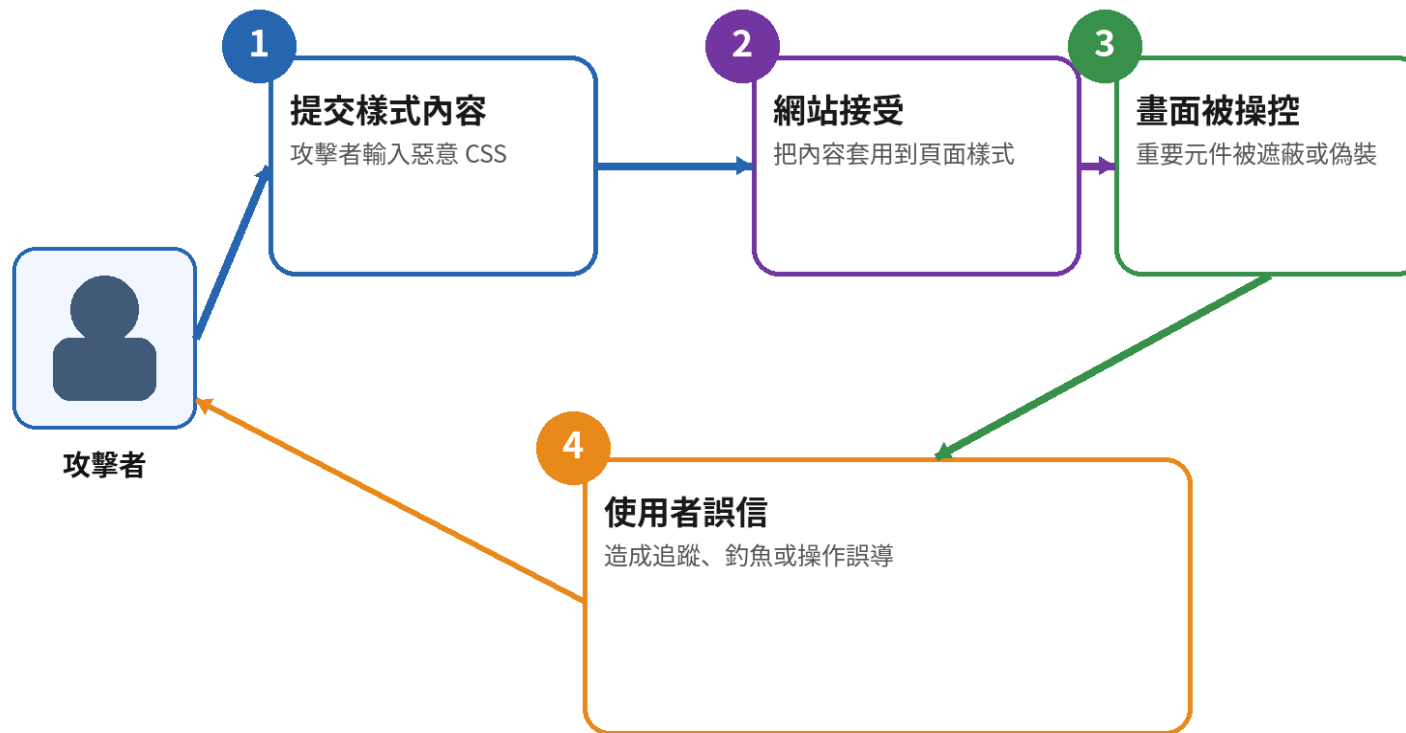
自訂主題、HTML email、富文字內容、
內嵌 style。

- **可能影響：**

畫面遮蔽、釣魚介面、資源載入追蹤，
甚至輔助資料**外洩**。

- **防護重點：**

限制可接受樣式、使用 allowlist、避免
未信任資料進入 style。



★ **重點：** 沒有 JavaScript 不代表無害，CSS 仍可能**改變**使用者所見與所做。

CRLF Injection | 換行字元可把資料變成新的 HTTP 標頭

實際情境

redirect 參數直接放入
Location ; %0d%0a 插入換行
後，可追加 Set-Cookie 等標頭
。

如何驗證

使用 ZAP Requester 檢查 Response Headers
是否出現非預期欄位或回應分裂。

安全改善

禁止 CR / LF、使用框架安全 Header API，並
以允許清單限制重新導向目的地。

修補證據

修補後回應只有預期標頭，輸入無法建立新行
。

CRLF Injection 是什麼？

- **定義：**

在輸入中插入換行字元，讓資料被解讀成新的 HTTP **Header** 或內容。

- **常見情境：**

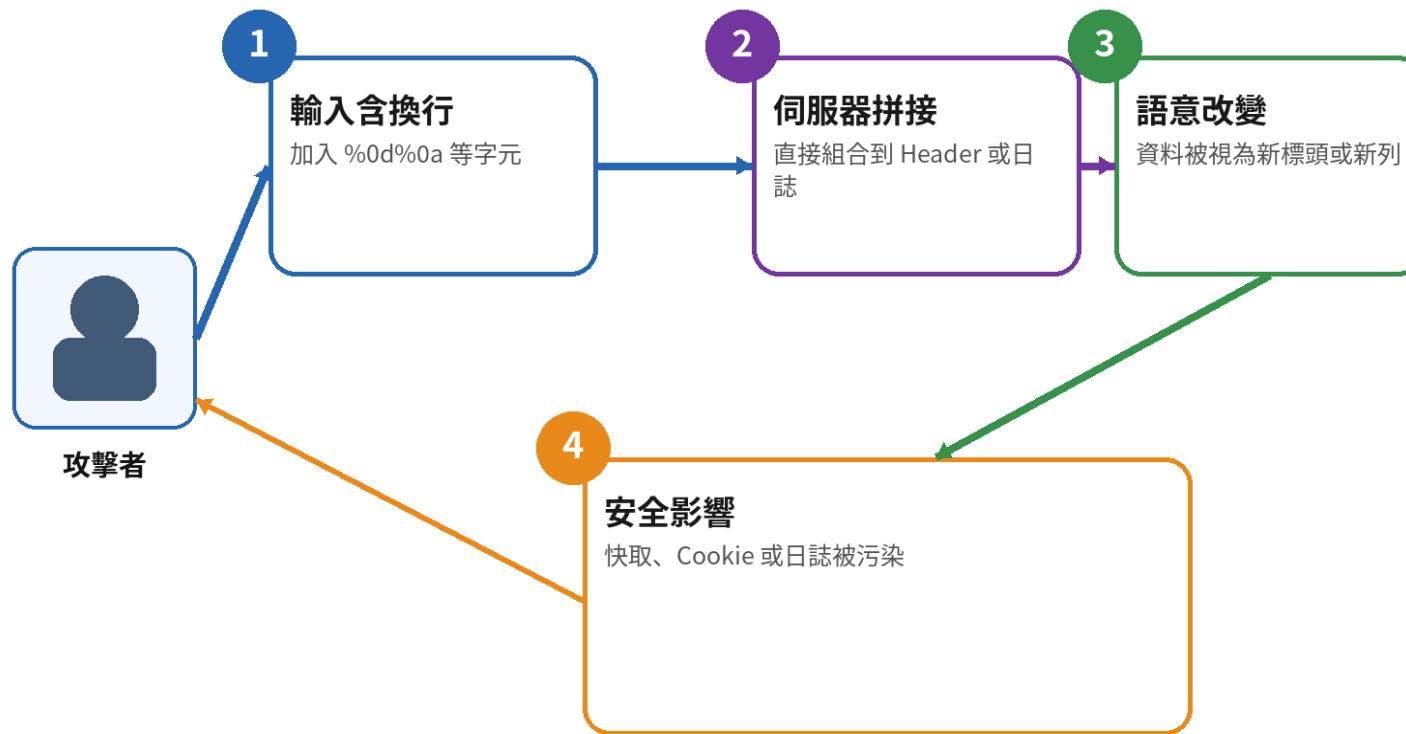
Location、Set-**Cookie**、**日誌**寫入、代理標頭。

- **可能影響：**

Header 污染、快取投毒、偽造回應或**日誌**。

- **防護重點：**

過濾 CR/LF、固定 **Header** 模板、避免**直接**拼接回應標頭。



★ **重點：** CRLF 注入不是改畫面，而是**改變**通訊與紀錄的邊界。

Eval Code Injection | 把字串交給 eval()，等同提供程式執行入口

實際情境

公式計算器直接 `eval("return " . $expr)`，攻擊者可插入 `system()` 等程式碼。

如何驗證

檢查 `eval`、`assert`、動態 `include`、反射與樣板解譯器的輸入來源。

安全改善

移除 `eval`，改用專用解析器；若僅允許算式，建立明確語法與字元允許清單。

修補證據

修補後英文字母、函式呼叫與控制語法均被拒絕。

Eval Code Injection 是什麼？

- 定義：

把未信任字串交給 `eval()`、`Function()` 等動態執行。

- 常見情境：

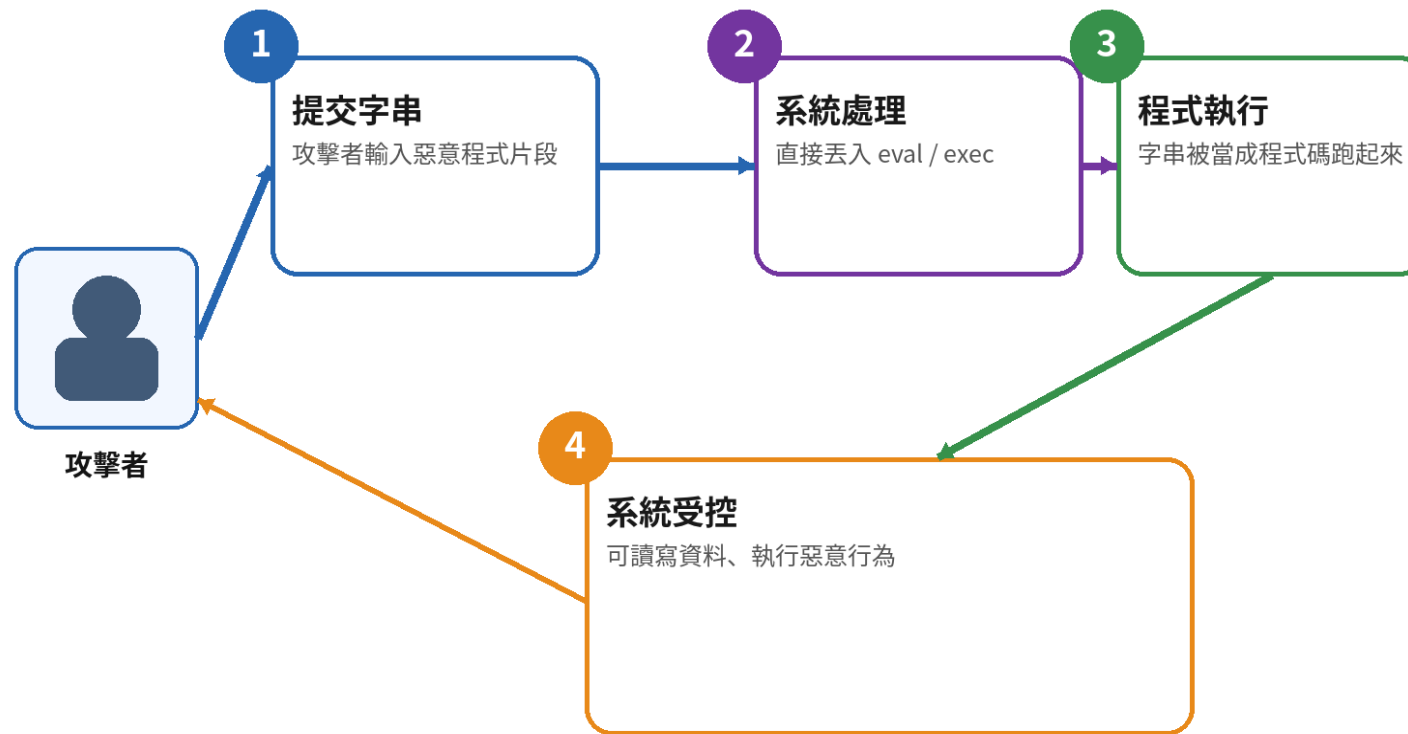
管理規則、自訂運算式、腳本功能直接 `eval`。

- 可能影響：

輸入變成可執行程式，風險接近遠端程式碼執行。

- 防護重點：

避免 `eval` 類 API，改用安全解析器或明確 allowlist。



★ **重點：** 只要把字串當程式跑，等於主動替攻擊者開了一扇門。

XPath Injection | XML 查詢字串也有資料與指令邊界

實際情境

帳號被直接拼入 XPath ; ' or 1=1 or ' 改變條件並回傳所有節點。

如何驗證

比較正常帳號、引號、布林條件與回應內容差異。

安全改善

使用支援變數綁定的 XPath API ; 限制輸入格式並避免動態組查詢。

修補證據

修補後輸入僅視為字串值，不再改變 XPath 結構。

XPath Injection 是什麼？

- **定義：**

當輸入被拼進 XPath 查詢，資料就可能 **改變** XML 查詢邏輯。

- **常見情境：**

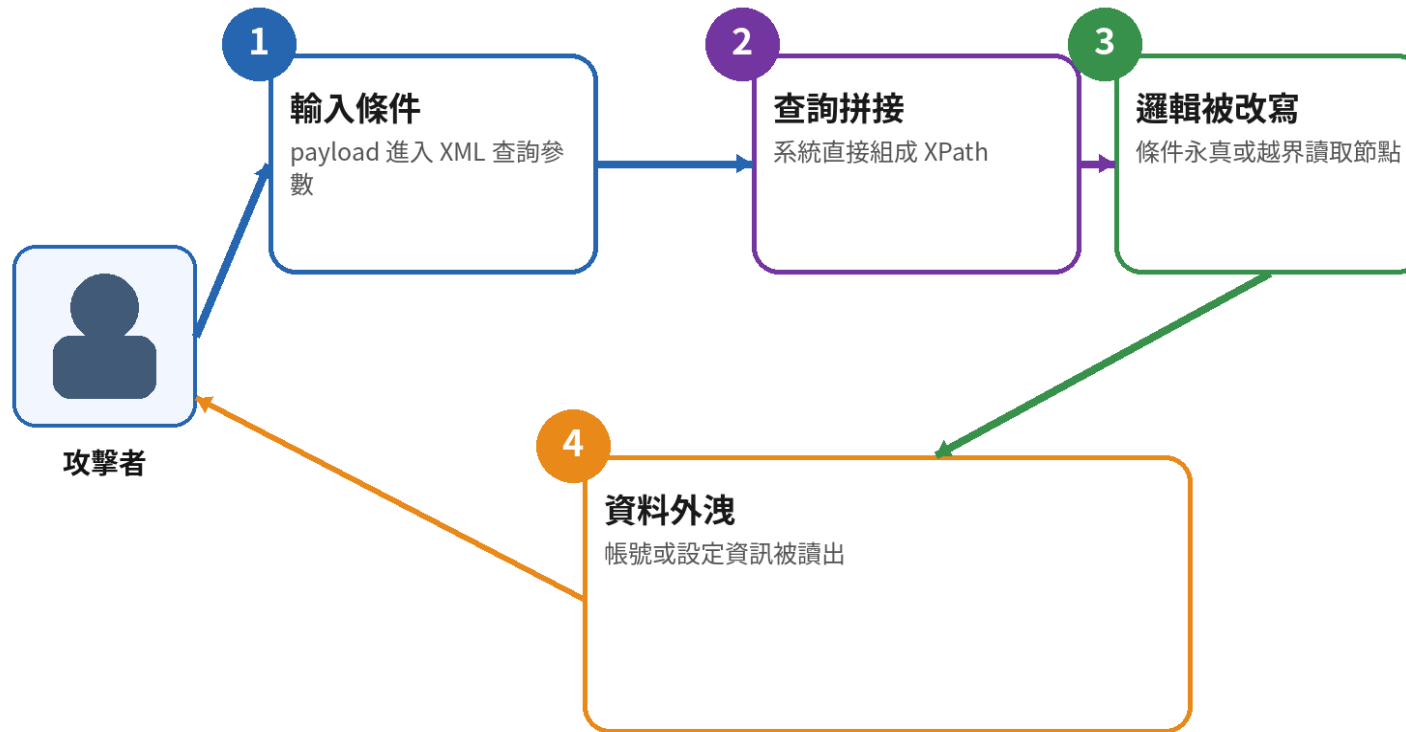
XML 使用者資料、設定檔、SSO 或舊系統查詢。

- **可能影響：**

繞過驗證、讀取額外節點或**敏感** XML 內容。

- **防護重點：**

使用安全 API / **參數化**機制、限制查詢模板與**輸入驗證**。



★ **重點：** 只要有查詢語言，就有資料與指令邊界被打破的風險。

LFI / RFI | 使用者控制 include 路徑，可能讀檔或執行外部程式

實際情境

`module` 參數直接交給 `include` / `require`，攻擊者選擇本機檔案或遠端來源。

如何驗證

檢查 `../`、絕對路徑、Wrapper 與遠端 URL；不得在非授權環境測試。

安全改善

以固定識別碼映射允許模組，停用不必要的遠端引入能力。

修補證據

修補後只有核准模組可載入，輸入不得轉成檔案路徑。

LFI (本機檔案包含) 是什麼？

- **定義：**

使用者可**控制** include/read 的本機檔案**路徑**。

- **常見情境：**

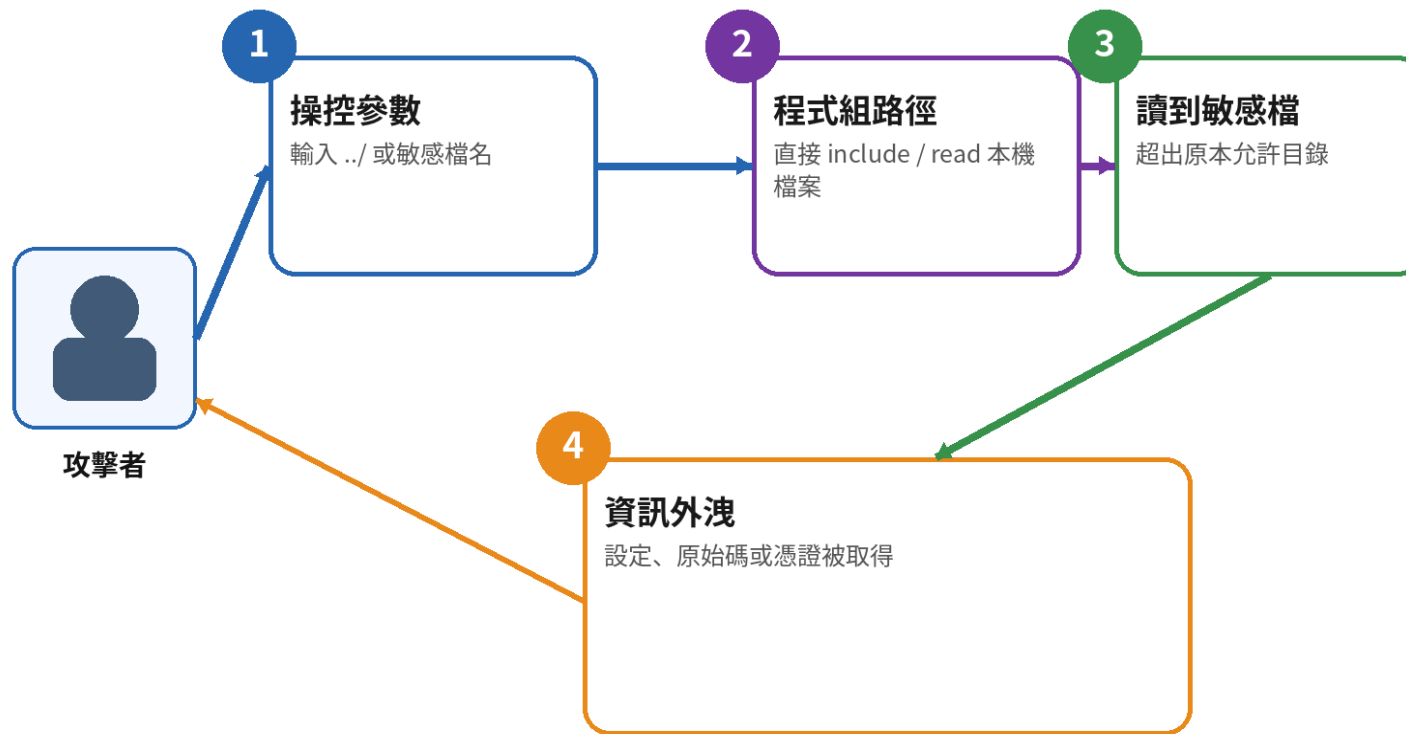
page=about.php、
module=news.php、readfile(file)。

- **可能影響：**

讀取設定、**原始碼**、**session**、log，甚至配合其他弱點執行程式。

- **防護重點：**

伺服器端 allowlist、固定映射、不接受任意**路徑**。



★ **重點：** LFI 的核心不是 include 本身，而是「**路徑**可被使用者**控制**」。

RFI (遠端檔案包含) 是什麼？

- **定義：**

系統允許以 URL 等遠端來源做 include / require。

- **常見情境：**

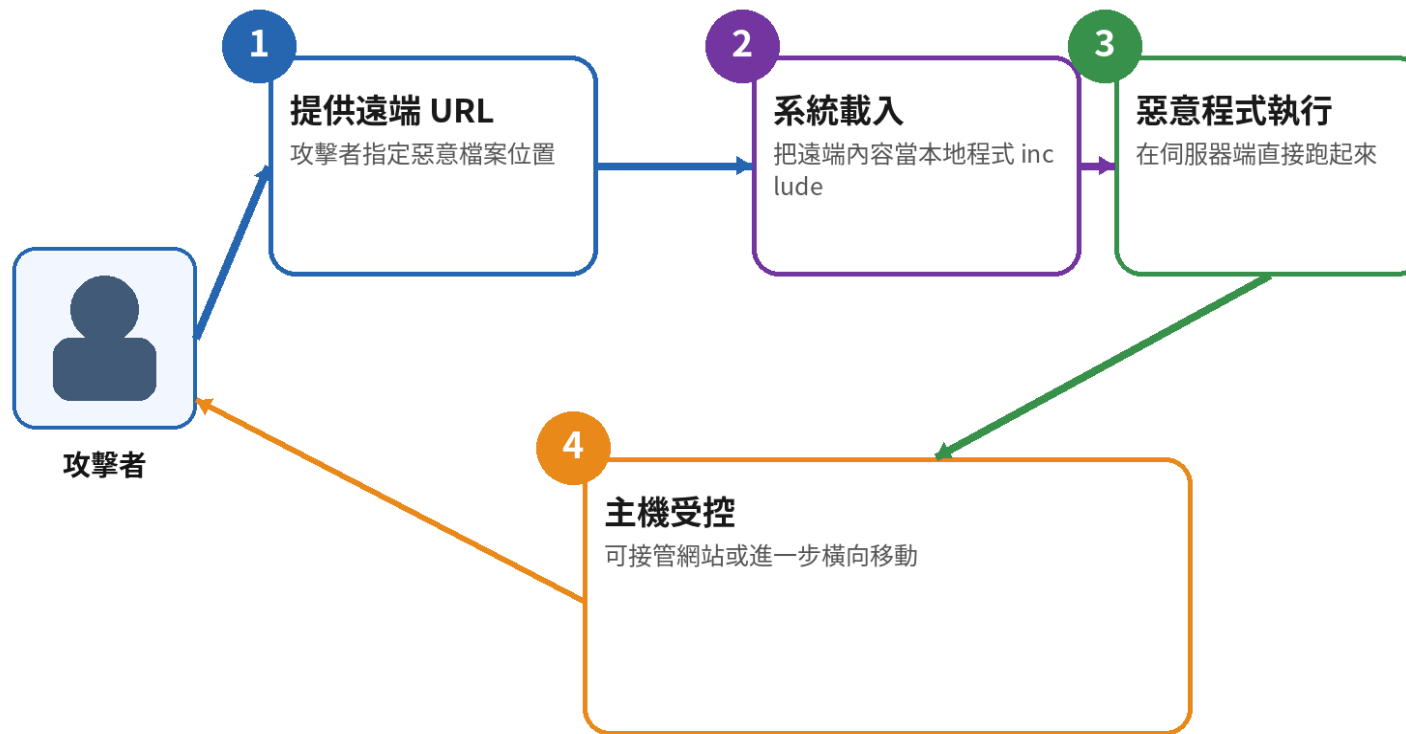
include(\$_GET["page"]); 且允許 http(s) wrapper。

- **可能影響：**

直接載入**攻擊者控制**的程式，風險接近遠端程式執行。

- **防護重點：**

停用危險 wrapper、限制來源、改用固定本機映射。



★ **重點：** RFI 比 LFI 更危險，因為它可能直接把外部程式帶進來執行。

其他注入類型 | 概念相同，解譯器不同

類型	解譯器 / 目標	主要防護	教材定位
NoSQL Injection	MongoDB 等查詢物件	型別驗證、禁止操作符注入、安全 API	列表補充
LDAP Injection	LDAP Filter	參數化 / 專用跳脫、允許清單	列表補充
ORM / HQL Injection	ORM 查詢語言	參數綁定，勿信任 ORM 自動安全	列表補充
EL / OGNL Injection	Expression Language	禁止不可信任表達式、升級框架	列表補充
SSTI	伺服器端樣板引擎	資料與樣板分離、Sandbox	列表補充
XML / XXE	XML Parser / 外部實體	關閉外部實體與 DTD、限制網路	可搭配 Lab
Prompt Injection	LLM 指令與工具鏈	信任邊界、輸出驗證、最小工具權限	延伸議題

A06

Insecure Design

程式即使沒有語法錯誤，流程與商業規則仍可能不安全

- 常見問題是流程跳步、交易上限缺失、重複領取與競爭條件
- 以威脅建模、安全需求、狀態轉換與失敗安全降低風險
- 自動掃描難以理解「正確業務結果」，必須設計濫用案例

A06 | VulnCampus 的商業邏輯與設計案例

負數報名

quantity=-1 造成負金額或名額增加

併發超賣

檢查名額與扣減未使用同一筆交易

優惠券重複使用

缺少使用次數與帳號限制

流程跳步

直接呼叫後續 API 略過前一步驗證

資源耗用

上傳檔案無大小或頻率限制

敏感流程無再驗證

高風險操作只依既有 Session

A06 案例 | 負數、超賣與優惠券濫用要一起驗證

實際情境

只做前端 $\text{min}=1$ ；後端未檢查範圍，也沒有交易鎖定與優惠券原子更新。

如何驗證

攔截修改 quantity、重複使用優惠券並以多個並行請求觀察名額及金額。

安全改善

伺服器端驗證型別與範圍；再以交易、FOR UPDATE / 原子更新及唯一約束維持一致性。

修補證據

不論單次或併發，金額、名額、優惠使用次數都維持一致。

負數 / 異常數量驗證缺失 是什麼？

- **定義：**

只在前端限制 $\text{min}=1$ ，後端卻未驗證數量範圍。

- **常見情境：**

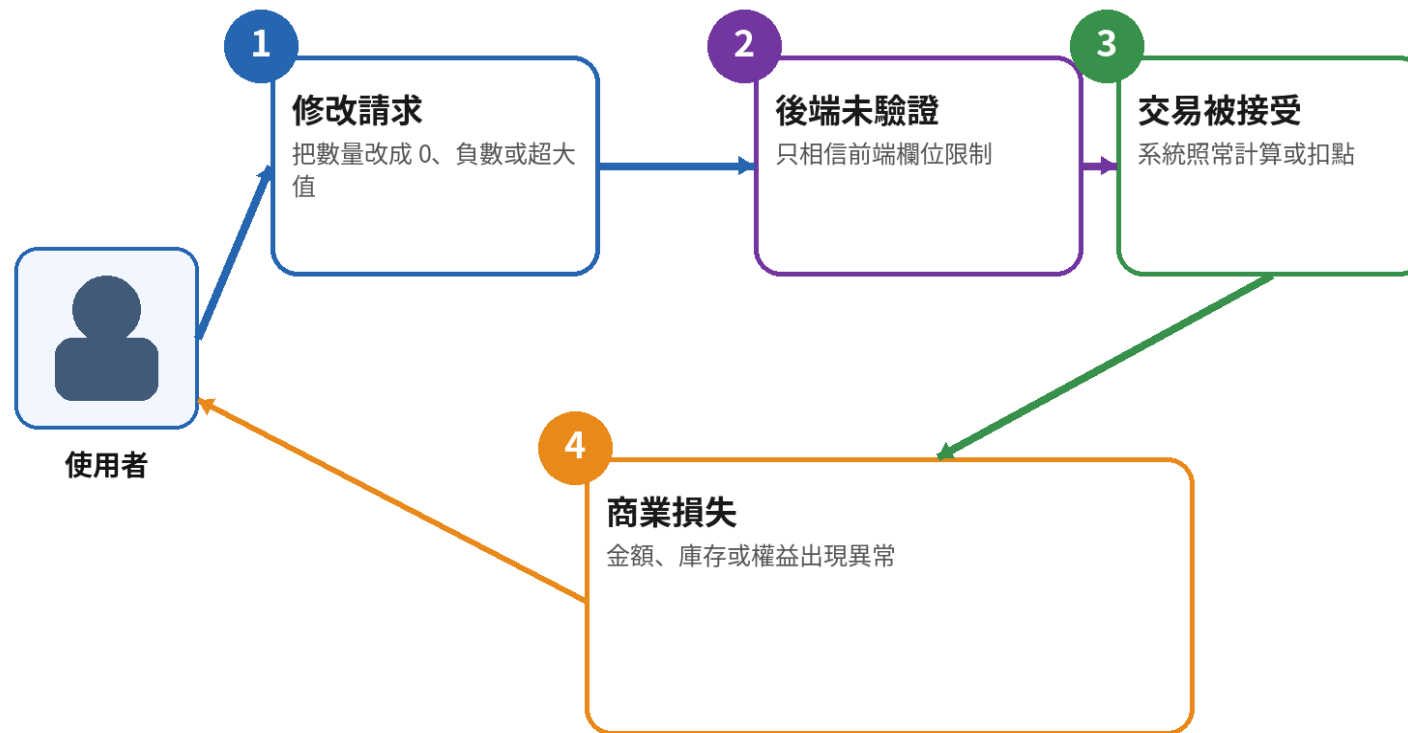
購物車 quantity、點數兌換、退款金額。

- **可能影響：**

可用負數抵扣、創造異常餘額或繞過商業限制。

- **防護重點：**

伺服器端檢查型別、範圍、上下限與業務規則。



★ **重點：** 商業邏輯不能只靠前端控制元件來保護。

超賣 / 競態條件 是什麼？

- **定義：**

多個請求同時搶同一資源，若缺少鎖定與一致性就會超賣。

- **常見情境：**

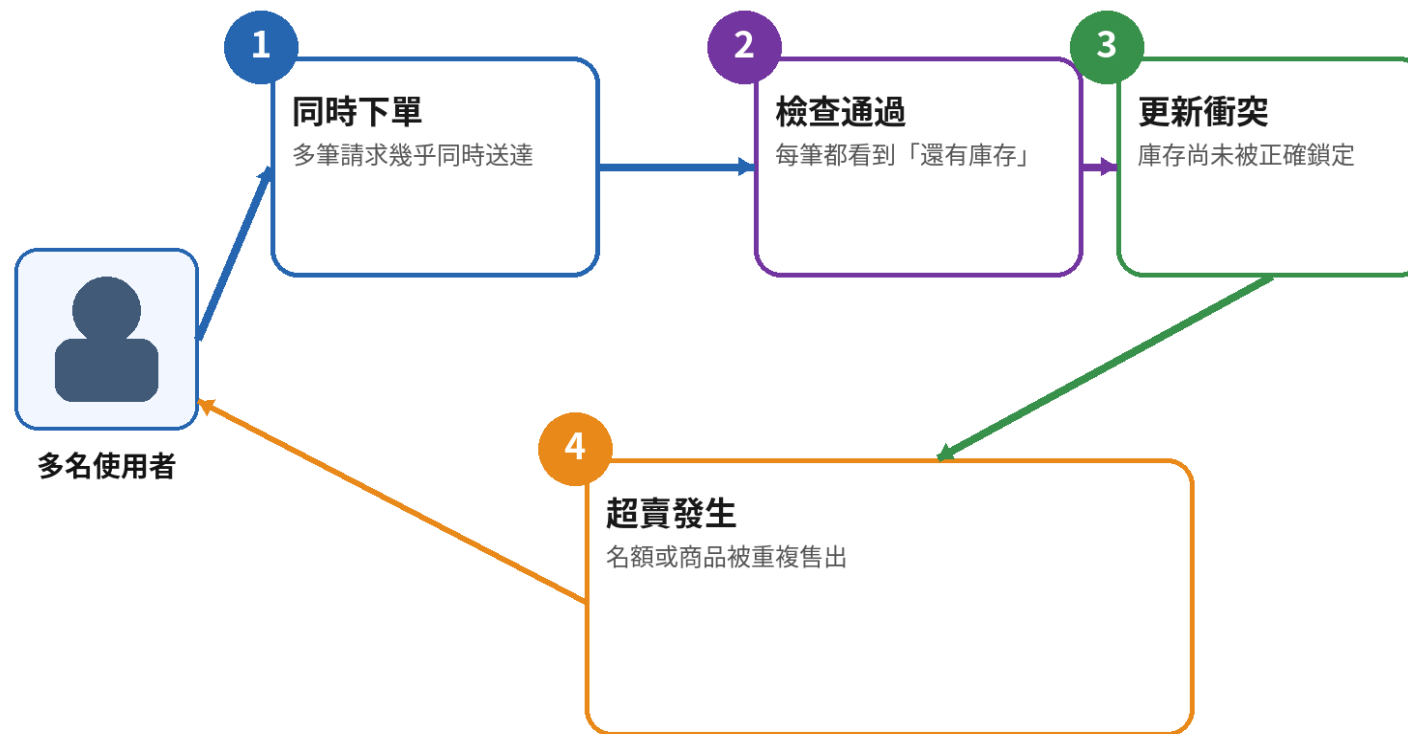
限量商品、活動名額、單次優惠。

- **可能影響：**

庫存負數、名額超額、交易狀態不一致。

- **防護重點：**

交易鎖、原子更新、唯一約束與重試 / 補償機制。



★ **重點：** 競態問題不是偶發小 bug，而是可被刻意利用的設計缺陷。

優惠券濫用 是什麼？

- **定義：**

優惠券使用條件、次數或綁定規則驗證不足。

- **常見情境：**

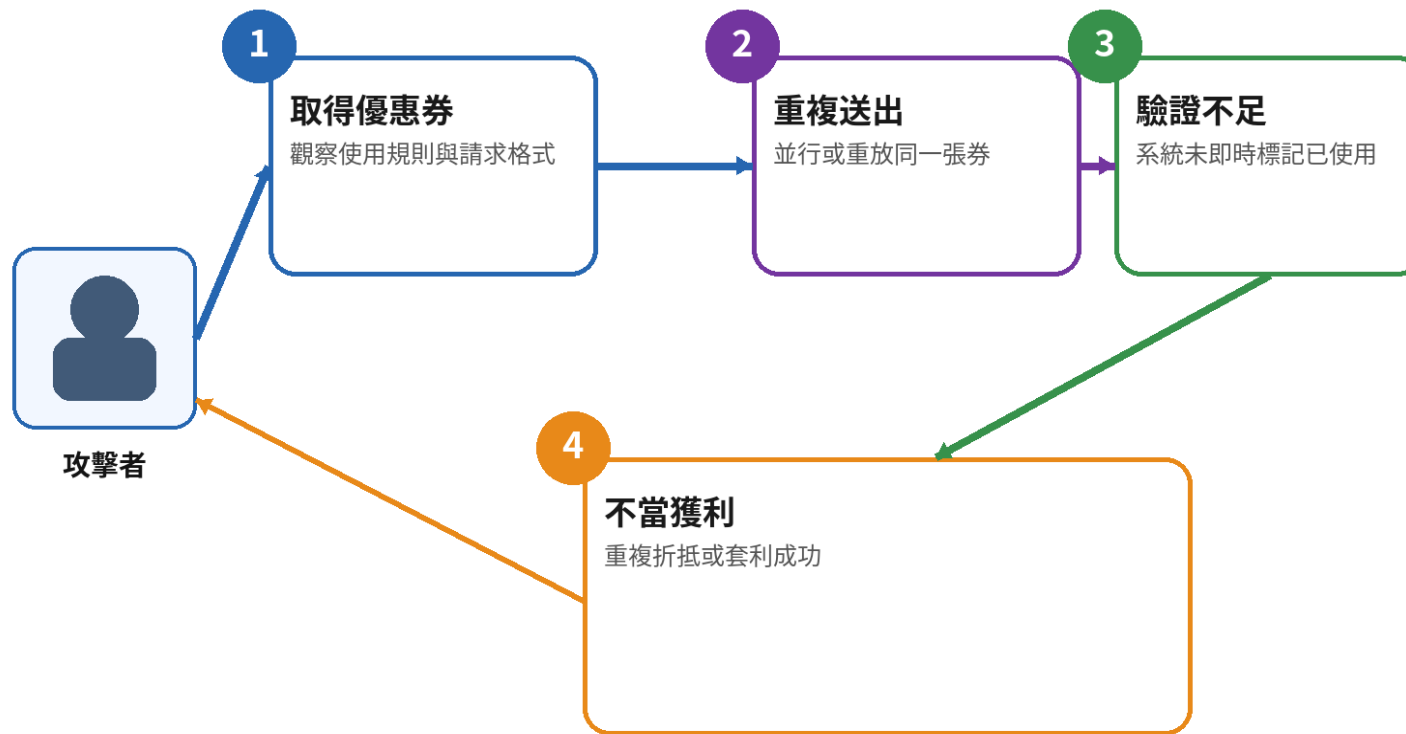
重複提交、共享代碼、多**帳號**套利、並行使用。

- **可能影響：**

折扣被**重複**套用，造成財務損失或活動失真。

- **防護重點：**

伺服器端驗證狀態、綁定使用者 / 訂單，並採原子更新。



★ **重點：** 優惠券不是前端顯示問題，而是完整交易規則與狀態管理問題。

A07

Authentication Failures

攻擊者可能冒用帳號、接管工作階段或繞過驗證流程

- 檢查登入、重設密碼、MFA、Cookie、登出與工作階段生命週期
- 驗證訊息、速率限制與 Token 設計都會影響安全性
- 改善採成熟身分服務、MFA、強密碼與安全工作階段管理

A07 | VulnCampus 的身分驗證案例

弱密碼

admin / admin、
password123

帳號列舉

錯誤訊息可分辨帳號是否存在

暴力破解

無失敗次數、速率限制或驗證碼

Session Fixation

登入前後 Session ID 未更新

重設密碼 Token

可預測、未綁定帳號、永久有效或可重放

閒置逾時

Session 長期不失效

A07 案例 | 登入成功只是驗證流程的一小段

實際情境

弱密碼、帳號列舉、沒有速率限制，再加上登入後不更新 Session ID，會形成帳號接管鏈。

如何驗證

比較錯誤訊息、重複登入速度、Cookie 變化、登出與逾時行為。

安全改善

一般化錯誤訊息、速率限制、MFA、登入後重新產生 Session ID 並設定安全 Cookie。

修補證據

重測時無法列舉帳號、暴力嘗試受控，且登入前後 Session ID 不同。

弱密碼 風險 是什麼？

- 定義：

密碼過短、常見、可預測或與個資高度相關。

- 常見情境：

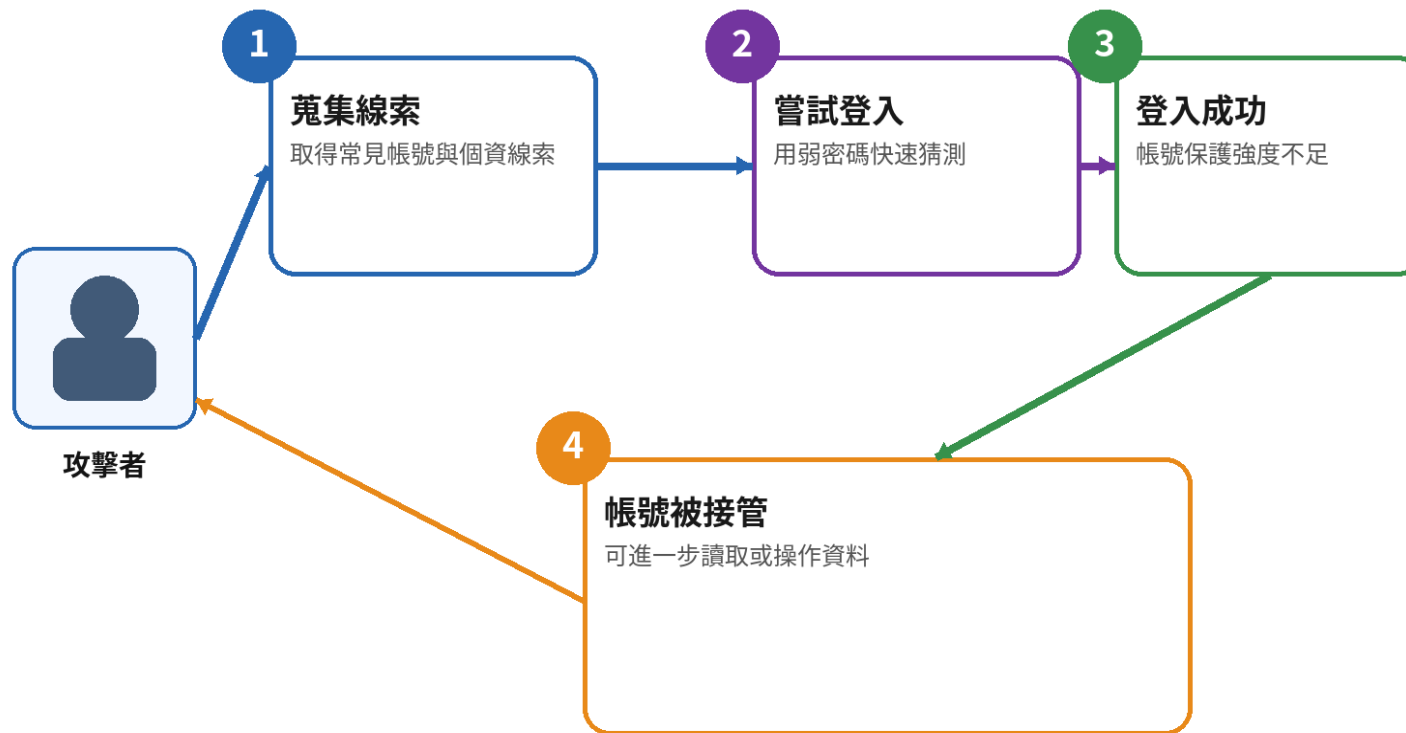
123456、生日、學號、公司名+年份。

- 可能影響：

撞庫、字典攻擊或社工猜測成功率大幅上升。

- 防護重點：

建立密碼政策、檢查常見弱密碼、支援MFA。



★ 重點：弱密碼讓後續所有安全控制都容易被繞過。

帳號列舉 是什麼？

- **定義：**

不同回應讓**攻擊者**分辨「**帳號**存在」或「**不存在**」。

- **常見情境：**

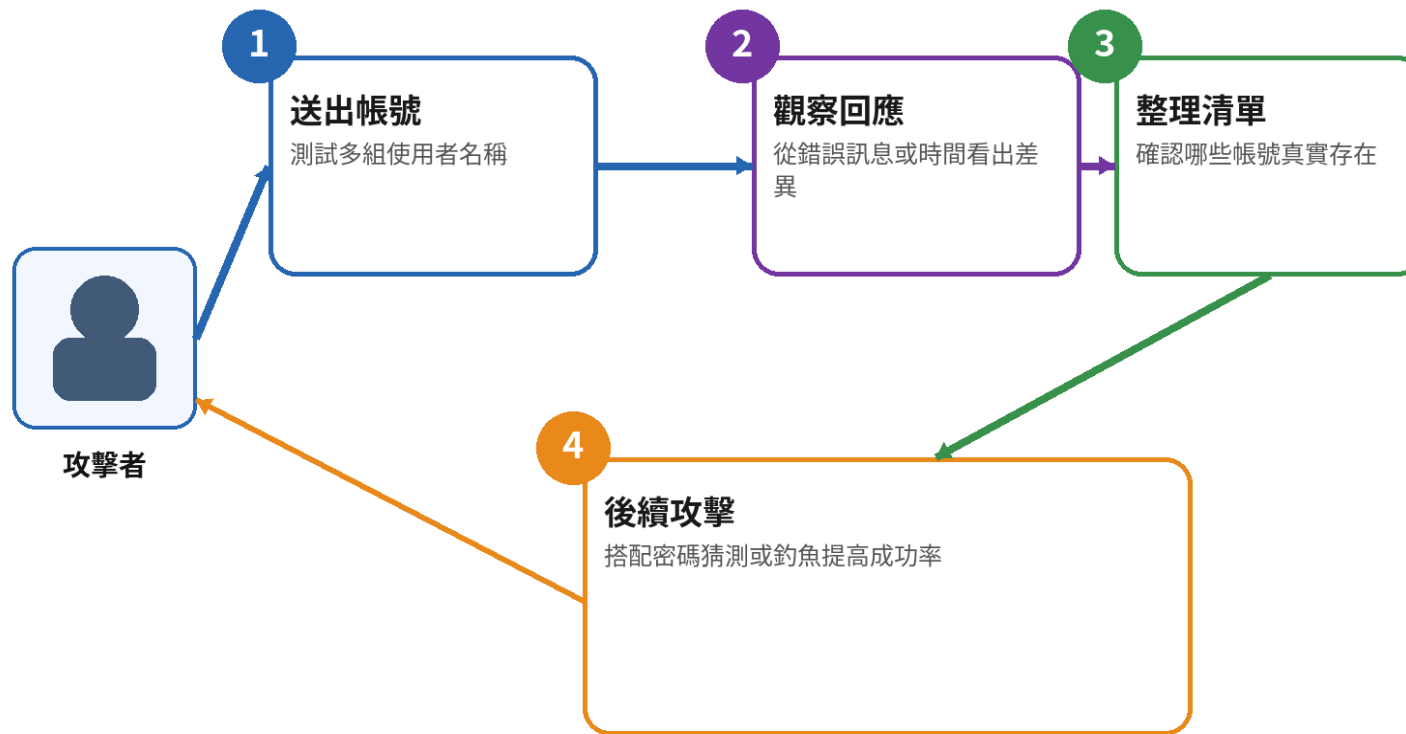
註冊、登入、忘記**密碼**、API 錯誤訊息差異。

- **可能影響：**

先整理出有效**帳號**清單，再搭配暴力破解或社工。

- **防護重點：**

統一回應訊息與流程，避免顯示過多差異。



★ **重點：** 列舉本身未必造成入侵，但它會大幅降低後續攻擊成本。

缺少速率限制 / 暴力破解 是什麼？

- **定義：**

登入、OTP、驗證碼或重設流程缺少嘗試次數限制。

- **常見情境：**

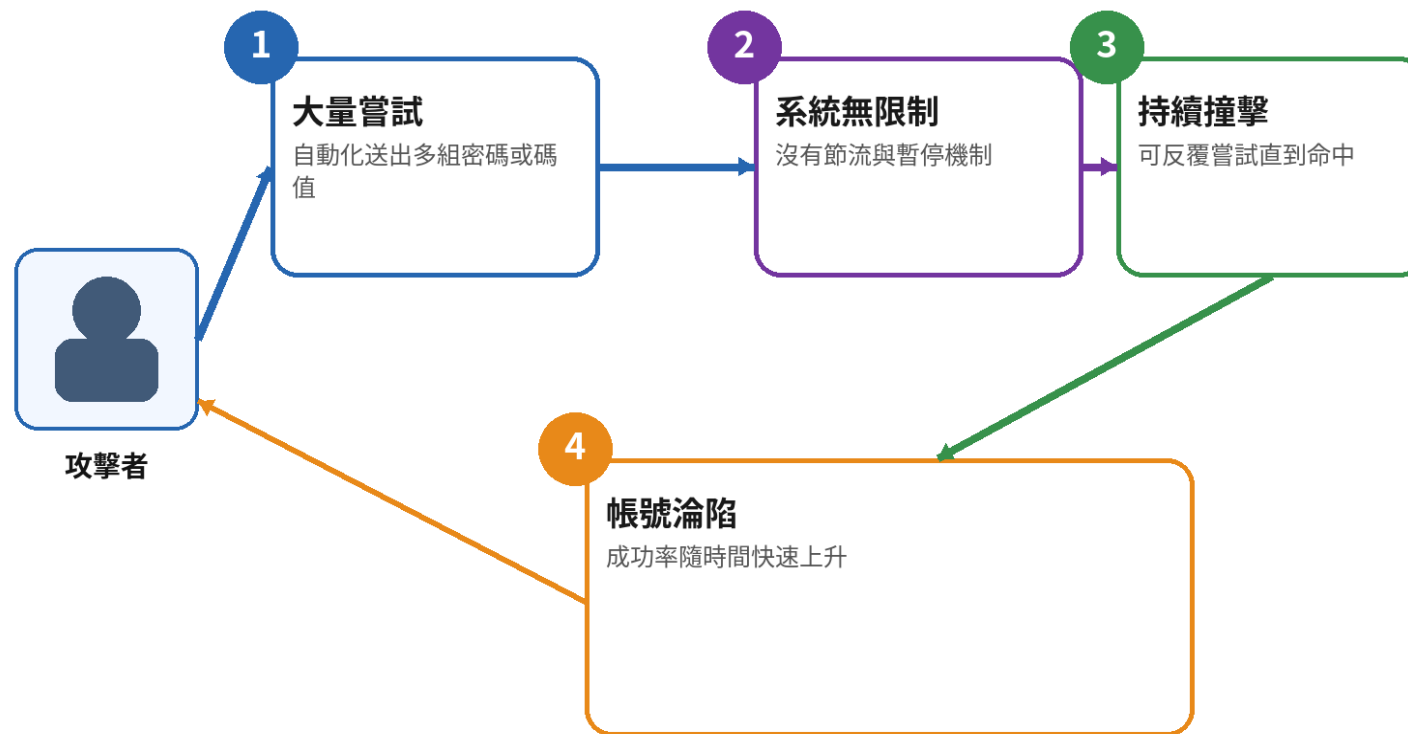
無 lockout、無節流、無 IP / 裝置監控。

- **可能影響：**

密碼猜測、驗證碼撞擊與自動化攻擊容易成功。

- **防護重點：**

節流、暫停、Captcha、MFA、異常偵測與告警。



★ **重點：** 沒有速率限制，登入頁就會變成攻擊者的猜密碼介面。

Session 固定 / 登入後未更新 Session ID 是什麼？

- **定義：**

登入前後沿用同一個 **Session ID**，讓已知 **Session** 可被接管。

- **常見情境：**

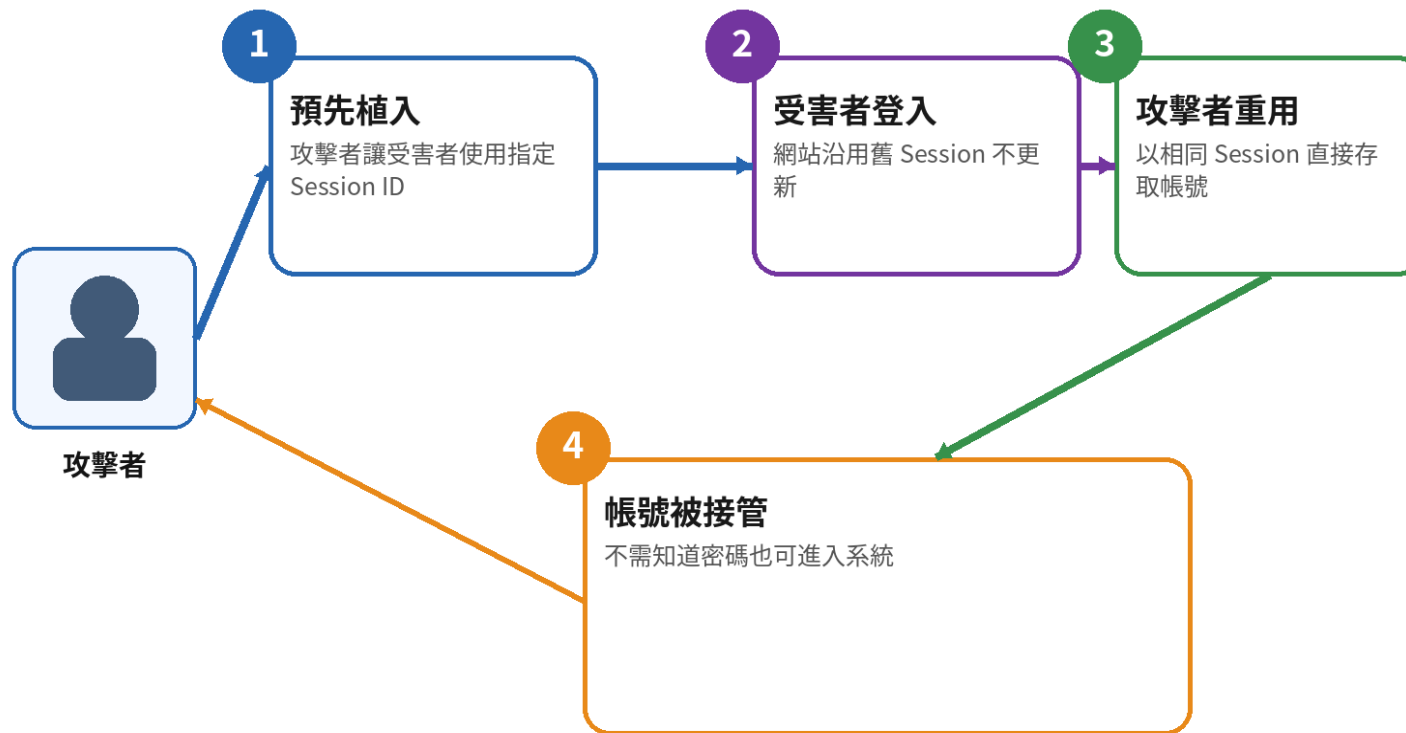
登入成功不 rotate **session**、URL / **cookie** 可預先植入。

- **可能影響：**

受害者一旦登入，**攻擊者**就能共用該 **Session**。

- **防護重點：**

登入後重發 **Session ID**、設定 **HttpOnly** / **Secure**、登出**失效**。



★ **重點：** 登入成功只是開始，**Session** 生命週期管理同樣關鍵。

A08

Software or Data Integrity Failures

系統信任未驗證的更新、外部資料、序列化內容或前端欄位

- 需確認來源、簽章、雜湊、資料模型與更新流程
- 改善採允許欄位清單、簽章驗證與可信更新來源
- 前端隱藏欄位不是完整性保護，使用者仍可修改請求

A08 | VulnCampus 的完整性案例

Mass Assignment

profile.php 接受 role=admin

不安全反序列化

unserialize() 載入攻擊者控制的物件

缺少 SRI

第三方 JavaScript 未驗證完整性

未簽章更新

下載檔案後未驗證來源或雜湊

隱藏欄位竄改

價格、角色或狀態由前端決定

CI/CD 內容竄改

建置產物缺少可追溯與簽章

A08 案例 | 隱藏欄位 role 被竄改為 admin

實際情境

表單雖隱藏 role，**伺服器**仍將所有欄位直接更新至資料庫。

如何驗證

先用瀏覽器開發者工具修改 hidden 欄位，再以 ZAP 攔截新增 role=admin，觀察伺服器是否接受。

安全改善

後端建立允許更新欄位清單；角色與權限完全由伺服器依身分決定。

修補證據

修補後即使請求含 role=admin，也被忽略或拒絕並留下稽核紀錄。

隱藏欄位 role 被竄改 是什麼？

- 定義：

把**權限**資訊放在前端隱藏欄位，使用者即可自行修改。

- 常見情境：

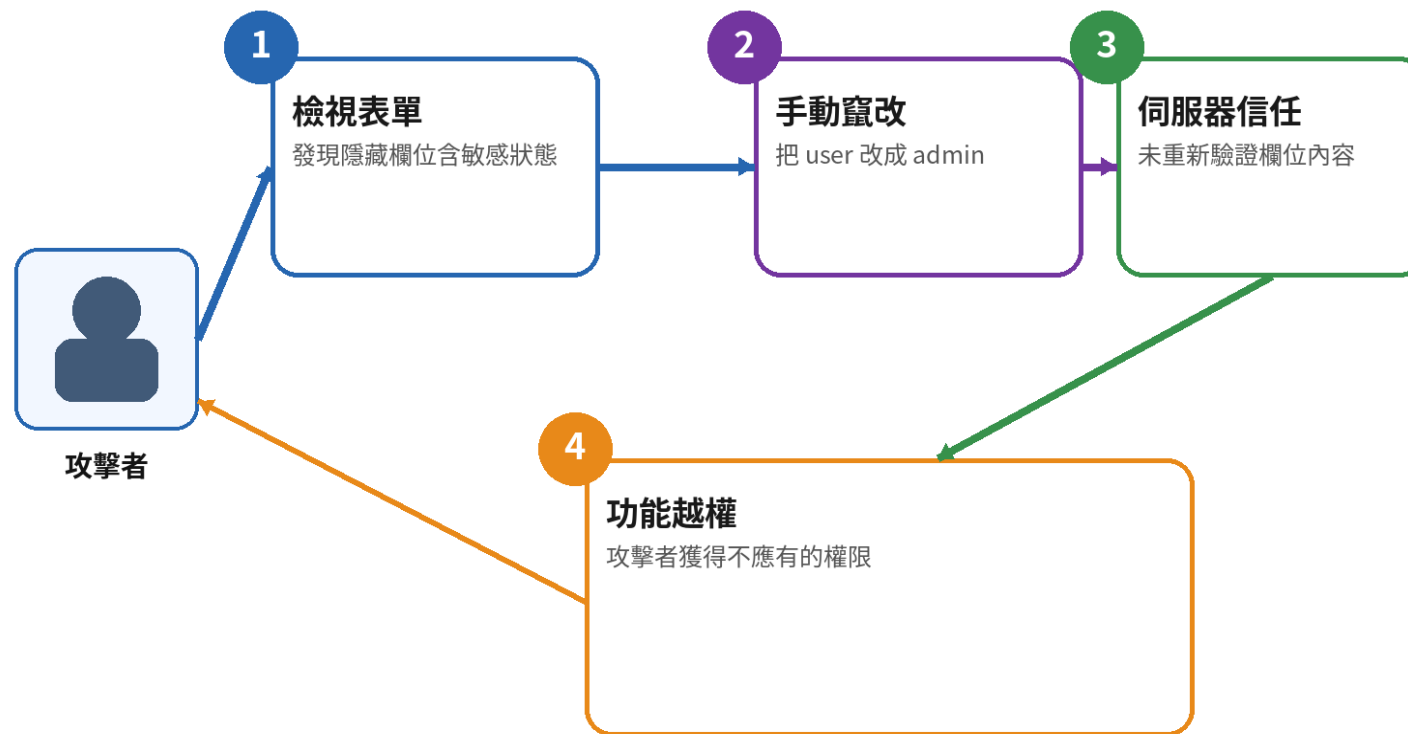
role=user、**price**=100、**isAdmin**=false 等隱藏欄位。

- 可能影響：

權限提升、價格竄改、流程**繞過**。

- 防護重點：

關鍵**權限**與價格一律由伺服器端重新計算與判定。



★ **重點：**隱藏欄位只是「隱藏」，不是安全**控制**。

A09

Security Logging and Alerting Failures

攻擊發生後無法及時發現、調查、通報或追蹤責任

- 只記錄正常交易不夠，異常與敏感操作更需要記錄
- 告警需定義門檻、責任人、通知管道與事件應變程序
- 日誌必須具備時間、身分、來源、事件、結果與關聯識別碼

A09 | 日誌與告警可用多種事件驗證

登入失敗

連續錯誤是否記錄並告警

權限變更

誰在何時將角色改為管理員

大量匯出

名冊筆數、操作者與來源

越權嘗試

IDOR、後台存取與拒絕結果

日誌含祕密

密碼、Token、Cookie 不得寫入

告警可達性

通知是否送達正確人員並可追蹤

A09 案例 | 有日誌但沒有告警，仍然無法及時回應

實際情境

系統只記錄登入成功；連續失敗、越權與大量匯出沒有事件或通知。

如何驗證

執行測試事件後檢查應用程式日誌、集中平台、告警與通知紀錄。

安全改善

定義安全事件、欄位、門檻與責任人；保護日誌完整性並定期演練。

修補證據

每個測試事件均能產生可關聯紀錄，達門檻時告警確實送達。

缺少安全日誌 是什麼？

- **定義：**

重要事件沒有被記錄，導致事後無法追查。

- **常見情境：**

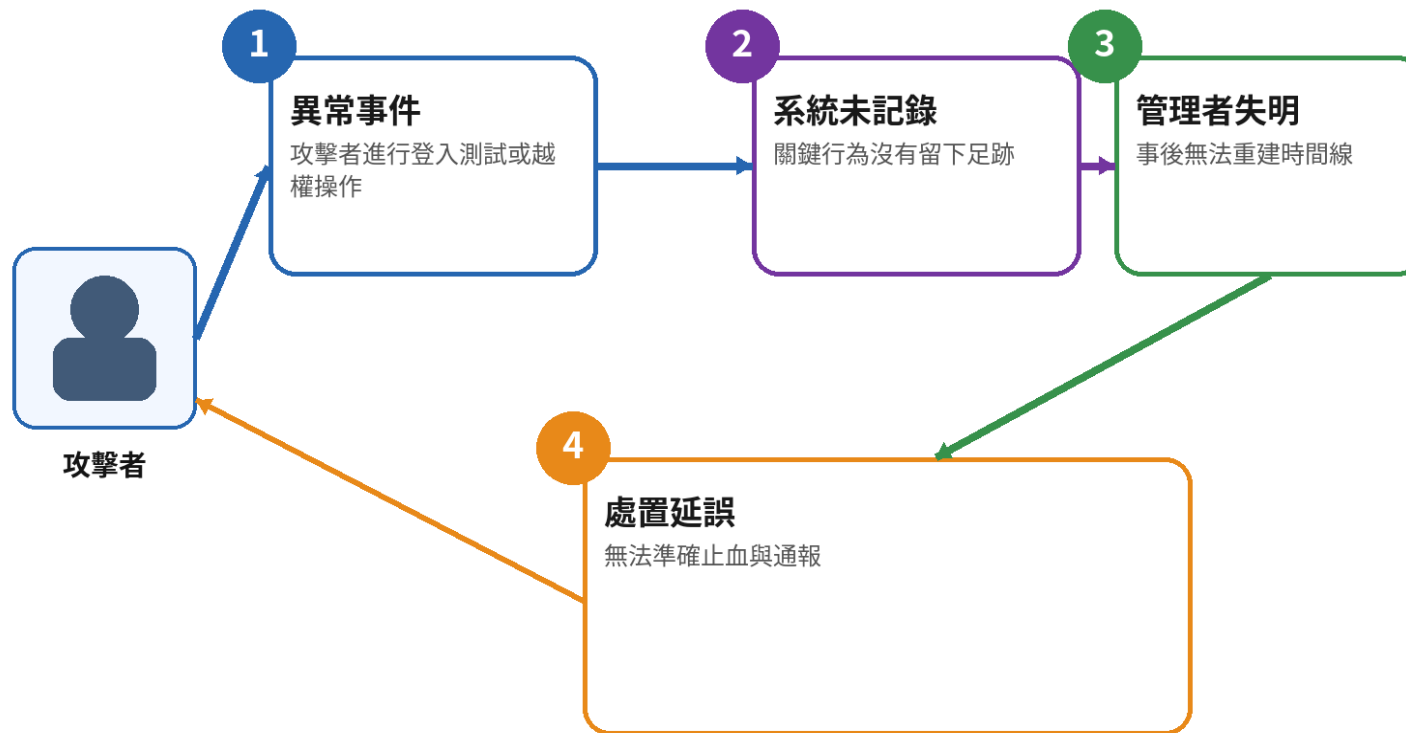
登入失敗、**權限**變更、**敏感**下載、管理操作未記錄。

- **可能影響：**

入侵後難以調查範圍、時間與受影響**帳號**。

- **防護重點：**

定義關鍵事件、記錄必要欄位、集中保存與防竄改。



★ **重點：** 沒有**日誌**，就像系統在黑暗中被攻擊。

有日誌但沒有告警 是什麼？

- **定義：**

事件雖有記錄，但無人即時發現與回應。

- **常見情境：**

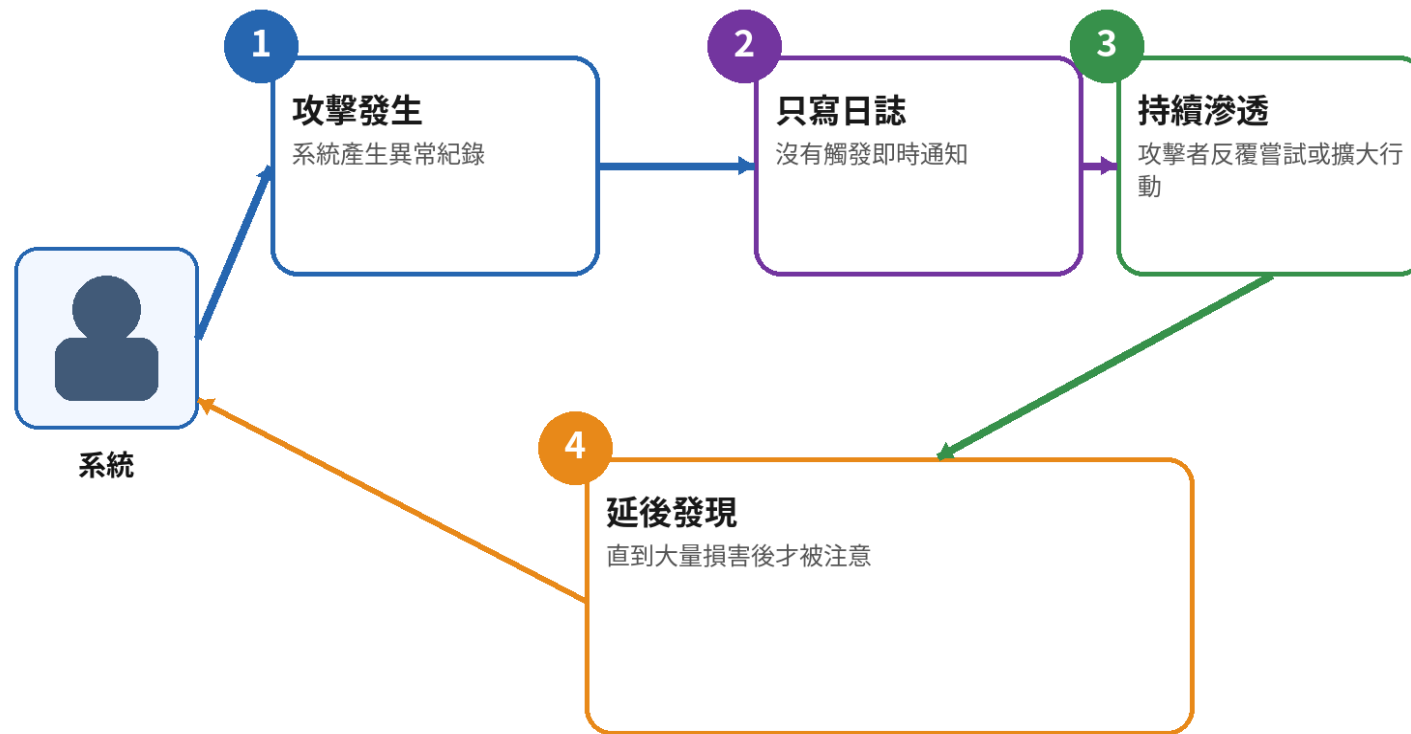
暴力破解、**權限**變更、異常下載只寫 log 不通知。

- **可能影響：**

攻擊可長時間持續，損害在發現前已擴大。

- **防護重點：**

建立**告警**門檻、SIEM / 郵件 / IM 通知與事件分級。



★ **重點：** 安全日誌要能被看見、被理解、被及時處置，才有價值。

A10

Mishandling of Exceptional Conditions

無效輸入、資源不足或相依服務失敗時產生不安全結果

- 常見為詳細 Stack Trace、狀態不一致、未回復交易或 Fail Open
- 改善採集中例外處理、安全錯誤訊息、交易回復與拒絕為預設
- 測試空值、超長值、型別錯誤、逾時、重複請求與部分失敗

A10 | VulnCampus 的例外狀況案例

Stack Trace 洩漏

`courses.php?q[]=X` 觸發
TypeError

SQL Exception

單引號造成資料庫錯誤直接輸出

Fail Open

驗證服務失敗後仍繼續敏感操作

部分交易失敗

扣款成功但名額或紀錄未完成

重複請求

相同操作被執行兩次

資源耗盡

超大輸入或大量上傳造成服務異常

A10 案例 | 錯誤時要安全失敗，而不是略過驗證

實際情境

資料庫或驗證服務拋出例外；
程式捕捉後仍繼續執行，或把
完整堆疊顯示給使用者。

如何驗證

模擬無效型別、逾時、斷線、重複請求與部分
交易失敗。

安全改善

集中處理例外、回復交易、使用關聯識別碼，
並採 Fail Closed。

修補證據

對外只顯示安全訊息；操作不生效且內部日誌
可完整追蹤。

Fail-open / 錯誤時安全失敗 是什麼？

- **定義：**

例外發生時若**直接**略過驗證或放行，系統就會 **fail-open**。

- **常見情境：**

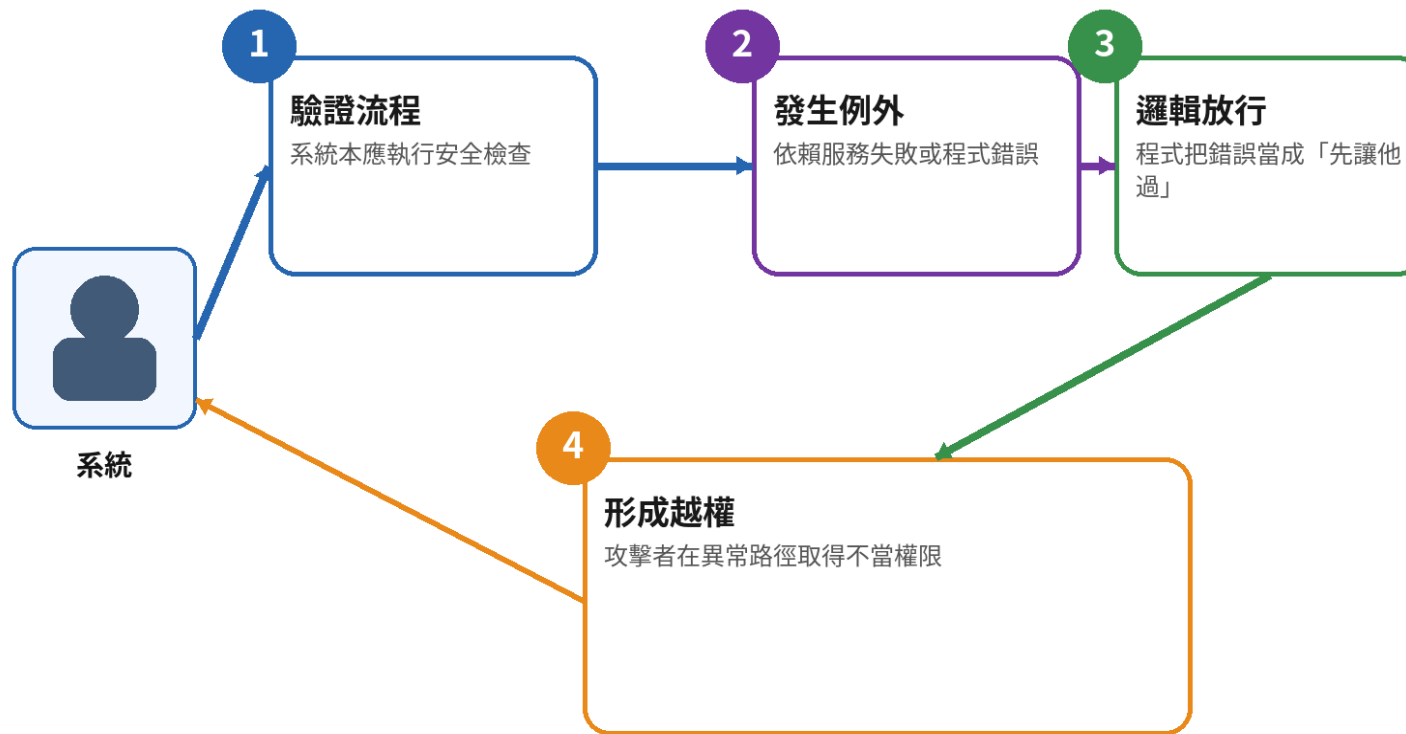
授權檢查出錯後 return true、第三方驗證失敗就略過。

- **可能影響：**

在異常情況下反而暴露更大**權限**與風險。

- **防護重點：**

預設**拒絕**、明確失敗、例外時**關閉**風險功能。



★ **重點：** 安全設計要 **fail closed**，不是 fail open。